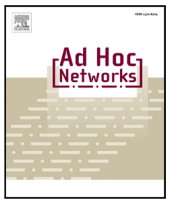
Ad Hoc Networks 181 (2026) 104065



Contents lists available at ScienceDirect

Ad Hoc Networks

journal homepage: www.elsevier.com/locate/adhoc





RELTO: A reliability-oriented DRL approach with context-aware adaptive reward weighting for multi-objective task offloading in MEC

Anam Nasir, Xiang He *, Teng Wang, Haomai Shi, Zhongjie Wang

Faculty of Computing, Harbin Institute of Technology, Harbin, 150001, Heilongjiang, China

ARTICLE INFO

Keywords:
Deep Reinforcement Learning
Multi-objective optimization
Quality of Service
Reliability-aware
Task offloading

ABSTRACT

Task offloading in Mobile Edge Computing (MEC) enables resource-constrained IoT devices to reduce latency and energy consumption while enhancing computational performance. However, designing effective offloading strategies presents a multi-objective optimization challenge, particularly in ensuring task reliability while optimizing energy efficiency and latency under dynamic conditions with unpredictable task failures caused by fluctuating computation demands and unstable communication links that severely degrade Quality of Service (QoS). Existing Deep Reinforcement Learning (DRL) approaches struggle to address these reliability-centered challenges, primarily due to their limited adaptability to dynamic reliability requirements, inadequate hybrid action space management, and insufficient handling of complex system state representations. To address these limitations, this work formulates a reliability-aware task offloading problem that explicitly integrates communication and computation reliability with latency and energy consumption into a multi-objective optimization formulation. To solve this optimization problem, the proposed Reliability Energy Latency Task Offloading (RELTO) algorithm employs Proximal Policy Optimization (PPO) within hybrid action spaces and incorporates a context-aware adaptive reward weighting mechanism driven by dual-attention architecture. The mechanism dynamically adjusts objective prioritization particularly emphasizing reliability based on real-time conditions, while attention-based state representation enables proactive decision-making through temporal pattern recognition. Extensive experiments in simulated MEC environments demonstrate that RELTO achieves optimal trade-offs across the key performance metrics, providing a more adaptive and robust solution for multi-objective task offloading.

1. Introduction

MEC is widely used to address the challenges posed by several computation and resource-intensive applications such as augmented or virtual reality (AR/VR), industrial IoTs, and autonomous systems by deploying computational resources closer to the end users [1,2]. Task offloading plays a crucial role in shifting these computations from the resource-constrained user devices to nearby edge servers. However, ensuring high QoS in such environments remains challenging due to the dynamic nature of transmission networks, workload demands and limited edge resources [3].

The task offloading process involves optimizing several key constraints, such as energy consumption, latency, and resource allocation, to maximize QoS [4]. Currently, the energy-delay trade-off has received significant attention and has been studied extensively [5], such as optimizing task scheduling to minimize energy consumption while maintaining acceptable delay [6], balancing computational load between edge servers to reduce overall energy usage and latency [7], and adjusting processing speed and power consumption according to user task requirements [8]. While reliability has emerged as a critical requirement alongside these traditional performance metrics and must be jointly optimized to ensure reliable task execution [9], it has received less attention and is often treated as a secondary or fixed constraint. Moreover, existing task offloading methods rarely adapt to the dynamic trade-off relationships among these multiple objectives in real-time environments.

In the context of MEC, reliability refers to the ability of the system to consistently execute tasks within the required time constraints [10]. Reliability is predominantly influenced by the instability of wireless communication channels, the variability of edge server computational capacity, and the dynamic and time-varying nature of task demands. To ensure reliable task execution, communication and computation reliability must be guaranteed.

Communication reliability is compromised due to signal quality fluctuations caused by degradation, interference, and congestion [1].

* Corresponding author.
E-mail addresses: anamnasir12441@gmail.com (A. Nasir), hexiang@hit.edu.cn (X. He), willtynn@stu.hit.edu.cn (T. Wang), shihaomai0109@163.com (H. Shi), rainy@hit.edu.cn (Z. Wang).

https://doi.org/10.1016/j.adhoc.2025.104065
Received 23 July 2025; Received in revised form 19 September 2025; Accepted 26 October 2025
Available online 30 October 2025
1570-8705/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

These fluctuations result in packet loss, high transmission delays, and retransmissions, increasing the likelihood of task failure. Low Signal-to-Noise Ratios (SNR) in dynamic environments exacerbate these issues, leading to poor transmission quality and high failure rates during task offloading [11].

Similarly, computational reliability is compromised when edge servers experience fluctuating computational resources due to varying workloads. When multiple devices offload tasks to the same edge server, overloading occurs, resulting in significant queuing and processing delays, frequently leading to task failures when demands cannot be met within stringent timeframes [12]. These computational failures not only violate latency requirements but also escalate energy consumption, thereby degrading the overall reliability.

Conventional approaches, such as heuristic algorithms, convex optimization and reliability block diagrams [13] have made progress in ensuring task reliability [12,14]. However, their reliance on static information makes them unsuitable for dynamic and partially observable settings and less effective in adapting to real-time variations in network and computation conditions [6]. Furthermore, these methods often treat reliability as a fixed constraint rather than an adaptive optimization goal, which limits their ability to prevent task failures under fluctuating conditions.

In order to address these limitations, DRL has emerged as a promising approach for creating adaptive task offloading policies in MEC [15–18]. DRL agents can adapt decisions based on real-time network and resource conditions [19]. However, two main challenges arise when using traditional DRL frameworks. First, task offloading involves both discrete and continuous decisions, requiring the agent to simultaneously manage hybrid action spaces. While some approaches focus on either discrete [20] or continuous spaces [21], others fail to effectively combine them, requiring complex transformations that limit the flexibility and expressiveness of the action space necessary to ensure reliability [22].

Second, as the task offloading is a multi-objective problem, the central challenge in applying DRL to such problems lies in reward scalarization [23]. Such as transforming the multiple objectives into a single scalar reward signal for the agent learning process [24]. Optimizing such complex rewards is challenging due to the presence of local optima, where policies might satisfy only a subset of objectives without considering others [25].

Conventional DRL approaches rely on static linear scalarization with predefined or fixed weights [9,17,26], which not only demand extensive manual tuning or prior domain knowledge, they also fail to adapt to the dynamism of MEC environments. For instance, prioritizing reliability during network congestion or energy efficiency when battery levels are low. These static approaches lead to rigid policies that cannot achieve optimal performance across diverse operational conditions. This problem is further compounded by the partial observability inherent in large-scale MEC systems, where factors such as communication delays prevent agents from having a complete, instantaneous view of the global state. This complex interplay of multi-objectives, inadequate scalarization, and state ambiguity due to partial observability profoundly hinders the development of truly intelligent, context-aware, and adaptive DRL mechanisms necessary for robust MEC optimization.

To address these challenges, this paper introduces a novel PPO-based algorithm that incorporates hybrid action spaces coupled with a context-aware adaptive reward weighting mechanism using dual attention to tackle partial observability of the DRL agent and extract the temporal dependencies for multi-objective optimization. This approach enables the agent to make both discrete decisions and continuous decisions simultaneously, dynamically adjusting the scalarization weights of the reward function based on the real-time state of the MEC environment. The agent learns when to prioritize latency, energy, or reliability, as the environment changes. This integration enhances the robustness of the offloading strategy in dynamic MEC environments. Following are the contributions of this study:

- We formulate a multi-objective optimization problem for a multi-user, multi-server MEC environment that explicitly captures the intricate interdependencies between reliability, energy consumption, and latency. To simplify the optimization process, we first linearize and relax the constraints, transforming the problem into a unified mathematical formulation that enables holistic QoS optimization. We then propose RELTO, that utilize Proximal Policy Optimization for policy learning and effectively manages a hybrid action space to enables fine-grained control over both the computation and communication reliability, enhancing the agent ability to make reliability-aware decisions in real-world MEC environments.
- We utilize a dual attention mechanism within the PPO framework to enable context-aware adaptive reward weighting. The feature attention component guides the agent to focus on critical state features such as task requirements, network conditions, and server load for effective decision-making. while the reward attention dynamically adjusts the scalarization weights of conflicting objectives in real time. This allows the agent to prioritize different objectives in real-time, such as focusing on latency during periods of high network congestion or prioritizing energy when the device battery is low. This adaptive reward weighting significantly enhances the agent ability to balance conflicting objectives while improving the QoS.
- We demonstrate the effectiveness of the proposed algorithm through extensive empirical evaluations to compare the performance across diverse scenarios. The comparative analysis with other baseline strategies demonstrates that the proposed framework achieves better reliability, lower energy consumption and latency in high-load environments while maintaining convergence stability under dynamic network conditions.

The remainder of this paper is structured as follows: Section 2 summarizes the related work, while Section 3 outlines the system model and formally defines the optimization problem. In Section 4, we provide a detailed explanation of the proposed method. Section 5 presents the simulation results to demonstrate the effectiveness of the algorithm. Section 6 concludes the paper.

2. Related work

This section presents the reliability-aware optimization of task offloading in MEC from both analytical and learning-based perspectives.

2.1. Traditional heuristic-based approaches

Among the previous literature, the heuristic-based approaches have been extensively utilized. Study [27] integrates Software-Defined Networking (SDN) with MEC to improve the reliability of computation offloading in IoV systems. Their approach uses a fault-tolerant particle swarm optimization algorithm to maximize reliability under latency constraints. The authors in [28] proposed the CR-BEAD framework to optimize edge application deployment based on coverage and reliability, focusing on resource management in small-scale deployments. While it addressed some aspects of reliability, it lacked the flexibility required for large-scale, real-time MEC environments. Study [29] proposed a multi-workflow scheduling algorithm for cloud systems, emphasizing reliability, cost, and energy efficiency, utilizing task duplication. However, the approach is limited to cloud-based workflows with a fixed scheduling policy and scalability issues. Study [30] combines stochastic network calculus to manage data flows and quantify QoS violations. The problem is formulated as a non-linear, non-convex mixed integer programming and is solved using a dual sub-gradient method for user association and a Newton–Raphson-based approach for power optimization and active caching among cells.

Similarly, the authors in study [31] propose a fast heuristic algorithm for optimizing energy consumption while ensuring reliability and latency constraints using Directed Acyclic Graph (DAG)-based subtasks to handle task dependencies and prioritize offloading decisions. Reliability is modeled against transient failures, while latency is managed with Earliest Start Time (EST) and Latest Finish Time (LFT) concepts. The method achieves lower energy consumption compared to local execution and random assignment. Study [14] focuses on maximizing admitted requests while ensuring reliability through VNF replication by utilizing integer linear programming for small-scale problems and a randomized approximation algorithm for larger instances. However, the randomized algorithm may lead to constraint violations, and the heuristic may be suboptimal compared to exact solutions. In the study [32], the authors propose a task-merging strategy for optimizing offloading points based on a shadow component scheme and formulate the problem as a combinatorial optimization and convert it into an n-fold integer programming problem. As it relies on approximations, which may not always guarantee optimal solutions. These heuristic-based approaches offer diverse solutions, however struggle with adaptability in real-time applications as they are designed with fixed assumptions and often fail to adjust to changing network conditions.

2.2. Reinforcement learning-based approaches

To address the challenges mentioned above, DRL methods are introduced for adaptive decision-making. The study [7] optimizes the task partitioning and computational allocation in a multi-user multi-server environment and formulate the optimization problem as a Markov Decision Process (MDP). A D3PG algorithm based on the Deep Deterministic Policy Gradient (DDPG) is proposed that utilizes the dirichlet distribution for continuous action space. Considering the energy-constrained UAV-assisted MEC server, the authors in study [8] propose a joint optimization strategy and model the problem as a partially observable MDP based on a multi-agent deep deterministic policy gradient. Similarly, in order to minimize the delay and energy consumption of intelligent vehicles the study [9] proposes an optimization framework based on the DDPG and blockchain. Additionally, taking into account the network and task characteristics, the study [26] proposed a hybrid task offloading method for Vehicular network systems. The DRL-based Twin Delayed Deep Deterministic Policy Gradient (TD3PG) algorithm was utilized for the computational offloading of the tasks.

Moreover, analyzing the joint optimization of energy and latency in a multi-user multi-server environment, the authors in study [33] utilize advantage Actor Critic (A2C) and compare it with other baseline algorithms. Study [10] presents two models, the Reliability Enhanced Task Offloading (RETO) and Differential Evaluation-based Task Offloading (DETO). A multi-objective optimization problem based on bandwidth and reliability model is formulated. Similarly, study [34] focuses on trajectory control and task offloading in a UAV-assisted MEC system. The authors proposed an Evolutionary Multi-Objective Reinforcement Learning (EMORL) model based on a multi-objective Markov decision process to solve task computation and offloading. The algorithm generates a set of non-dominated policies, requiring an additional step for final policy selection in real-time, which might not be adaptively optimized for dynamic reliability priorities. In the study [35] the authors utilize the Q-learning based optimization for the task offloading by minimizing the response time and propose a model Q-TOMEC. The system model comprised multiple users and servers. However, the system model is limited to only two servers. In [15] the authors presented a dynamic offloading problem for a multi-user MEC network where tasks can be partially offloaded to multiple Computational Access Points (CAPs). To optimize the task offloading strategy in time varying CAPs environment, the author formulates a Deep Q Network (DQN)-based offloading strategy.

The growing complexity of several components of service computing systems necessitates the accurate analysis and optimization of their reliability. Study [36] proposes a joint approach for reliability-aware data compression and task offloading, using Markov models for system dynamics and formulating a convex optimization problem to minimize latency while considering reliability and energy constraints. The solution was implemented through a distributed Reliability-aware Task Processing and Offloading (RTPO) algorithm. Similarly, for the reliability analysis, the authors in [37] used dynamic distribution parameters and proposed a fault-tolerant model for fog systems using a Markov chain and using Simulated Annealing (ISA) solutions. In study [38], the proposed approach incorporates a combinatorial multi-armed bandit (CMAB) integrated with Lyapunov optimization to address task scheduling for DNN inference tasks. The framework optimizes both the accuracy of the inference tasks and their reliability under uncertain network and edge server availability. However, high computational overhead makes the adaptation to real-time systems challenging. Similarly, study [22] proposed the A-MAHPPO algorithm. The authors integrate a novel reward design for infeasible actions and a hybrid action space handling mechanism, allowing for better coordination among multiple agents in a distributed MEC system.

In [39] the authors focus on developing a more stable and effective DRL-based offloading strategy for complex MEC tasks by addressing the instability and overestimation bias of earlier algorithms such as DDPG. The authors first utilize the Directed acyclic graph (DAG) with Graph neural networks (GNN) for state representation and then utilize TD3 with twin critics and single actor for the efficient task offloading. In order to demonstrate the potential of intelligent and real-time task allocation in MEC the study [40] utilize D4PG based algorithm where the agent learns an adaptive policy to make real-time decisions about where to process tasks to optimize system performance. To address the challenge of reducing latency in edge computing by intelligently offloading dependent tasks the study [41] utilizes PPO and transformer architecture. The transformer architecture is used to process the high-dimensional and relational state space of tasks and devices, PPO algorithm then learns a policy to make optimal offloading decisions to minimize latency. Study [42] proposes an enhanced Deep Deterministic Policy Gradient (DDPG) algorithm for task scheduling in Mobile Edge Computing (MEC) systems, focusing on reducing latency and energy consumption. The algorithm incorporates a soft update mechanism and an experience replay buffer to improve stability and efficiency. Similarly, to optimize the sequence of offloading and minimize the delay-energy tradeoff the authors in [43] utilize DAG to represent the tasks and then utilize sequence to sequence (S2S) neural network with SAC. The authors in [44] argue that existing AI-based solutions suffer from lack of adaptability to stochastic environments, which limits their practicality. Hence, the paper introduces a Decision Tree Empowered Reinforcement Learning (DTRL) framework. This method aims to design a unified framework for performing both computational offloading and resource allocation at runtime by leveraging the decision tree ability to make fast, explainable decisions while using reinforcement learning to adapt to the dynamic environment and optimize energy and latency.

While these studies offer diverse approaches, they often suffer from suboptimal state representation, failing to effectively process high-dimensional, structured observations and capture temporal dependencies, leading to ambiguous states. A primary limitation is the reliance on static reward scalarization for multi-objective optimization, which prevents dynamic adaptation of objective priorities in changing environments. Furthermore, many DRL algorithms struggle with hybrid action spaces, requiring complex transformations and suboptimal approximations. Table 1 provides a comparative analysis of the literature studies based on latency, energy consumption, reliability, and applied algorithms, respectively.

Table 1
Comprehensive comparison of the approaches in the literature

Ref.	Approach used	Architectural Properties		System Modelling		Optimization Techniques			Hybrid action space	Decision Objectives				Learned state representation	Context aware adaptive reward
		Edge and Cloud	Edge	SOO	MOO	Traditional	Single Objective RL	Multi Objective RL		Latency	Energy	Reliability			
											Comm.	Comp.			
[7]	D3PG	×	✓	✓	×	×	✓	×	✓	✓	✓	×	×	×	×
[8]	DDPG	×	✓	✓	×	×	✓	×	×	✓	✓	×	×	×	×
[9]	DDPG	✓	×	✓	×	×	✓	×	×	✓	✓	×	×	×	×
[26]	TD3	×	✓	✓	×	×	✓	×	×	✓	✓	×	×	×	×
[33]	A2C	×	✓	✓	×	×	✓	×	×	✓	✓	×	×	×	×
[10]	RETO/DETO	×	✓	×	✓	×	✓	×	×	✓	×	✓	✓	×	×
[15]	MDP(DQN)	×	✓	✓	×	×	✓	×	×	✓	✓	×	×	✓	×
[45]	MDP+ Greedy	×	✓	×	✓	×	✓	×	✓	✓	×	×	✓	×	×
[36]	MDP(RTPO)	×	✓	✓	×	✓	×	×	×	×	×	×	✓	×	×
[32]	Greedy Heuristic	×	✓	✓	×	✓	×	×	✓	×	×	×	✓	×	×
[37]	Fault-tolerant Heuristic	✓	×	×	✓	✓	×	×	×	✓	×	×	✓	×	×
[34]	DRL+Evolutionary	×	✓	×	✓	×	×	✓	✓	✓	✓	×	×	×	×
[19]	DQN	✓	×	✓	×	×	✓	×	×	✓	✓	×	×	✓	×
[22]	MADRL	×	✓	×	✓	×	×	✓	×	✓	✓	×	×	×	×
[27]	FPSO-MR	×	✓	✓	×	✓	×	×	×	✓	×	×	✓	×	×
[29]	DVFS	✓	×	×	✓	✓	×	×	×	×	×	✓	✓	×	×
[30]	SNC Heuristic	×	✓	×	✓	✓	×	×	✓	✓	×	×	✓	×	×
[31]	DAG+ Heuristic	×	✓	×	✓	✓	×	×	×	✓	×	✓	✓	×	×
[38]	RTBL	×	✓	×	✓	×	✓	×	×	×	×	×	✓	×	×
[43]	DAG+SAC	×	✓	✓	×	×	✓	×	×	✓	✓	×	×	×	×
[44]	DTRL	×	✓	✓	×	×	✓	×	✓	✓	×	×	×	×	×
[41]	TPTO	×	✓	✓	×	✓	×	✓	×	×	×	×	×	✓	×
[40]	D4PG	×	✓	×	✓	×	×	✓	✓	✓	✓	×	×	×	×
[39]	DAG+TD3	×	✓	✓	×	×	✓	×	✓	✓	✓	×	×	✓	×
[Ours]	PPO+Dual Attention	×	✓	×	✓	×	×	✓	✓	✓	✓	✓	✓	✓	✓

Table 2
Description of the parameters.

Symbol	Description	Symbol	Description
$\mathcal{U} = \{ue_1, \ldots, ue_m\}$	Set of user devices (U)	$\mathcal{E} = \{es_1, \ldots, es_n\}$	Set of edge servers (E)
μ_n, Q_n, M_n	Computational capability (μ_n), cores (Q_n) and memory (M_n) of E	Sc	Small cell where $\{es_n, ue_m\} \in Sc$
$\theta = \{\theta_1, \ldots, \theta_i\}$	Set of tasks	$\theta_{i,m}(t)$	Task i generated by U_m at time t
I_i	Input size of task i	C_i	CPU cycles required for task i
R_i	Number of cores required by task i	T_i	Time requirement (deadline) of task i
$L^{local}_{i,m}(t)$	Local computation latency of task i at U_m	$L^{off load}_{i,m,n}(t)$	Offloading latency of task i to E_n
$L^{trans}_{i,m,n}(t)$	Transmission latency of task i	$L^{prop}_{i,m,n}(t)$	Propagation latency of task i
$L^{queue}_{i,m,n}(t)$	Queuing latency of task i	$L^{comp}_{i,n}(t)$	Computation latency at En
$U_{i,m,n}(t)$	Uplink rate between U_m and E_n	b	Channel bandwidth
$\text{SINR}_{i,m,n}(t)$	Signal-to-Interference-plus-Noise Ratio	$d_{m,n}$	Distance between u_m and E_n
$P_{i,m,n}(t)$	Transmission power of U_m for task i to E_n	P_r, P_i, N_0	Received power, interference power, and noise power
$E^{local}_{i,m}(t)$	Local computation energy consumption	$E^{off load}_{i,m,n}(t)$	Energy consumption when offloading
$E^{comp}_{i,n}(t)$	Computational energy at E_n	$E^{trans}_{i,m,n}(t)$	Transmission energy for task i
$Re^{comm}_{i,m,n}(t)$	Communication reliability of task i	$Re^{comp}_{i,n}(t)$	Computation reliability of task i
λ	Failure rate during transmission	ϵ	Failure rate during computation
$Re^{off load}_{i,m,n}(t)$	Total reliability when task i is offloaded	$Re^{local}_{i,m}(t)$	Reliability when task i is executed locally
$\phi_{i,m}(t) \in \{0, 1\}$	Offloading decision (0: local, 1: offload)	$P_{\min}, P_{\max}$	Minimum and maximum transmission power limits
$L_{\text{total}}(t)$	Total system latency at time t	$E_{\text{total}}(t)$	Total system energy consumption at time t
$Re_{\text{total}}(t)$	Total system reliability at time t	α, β, γ	Weighting coefficients for latency, energy, and reliability

3. Problem formulation

This section presents the network model, the task model, task latency, energy model and the reliability model. Afterwards, the problem formulation is presented. Additionally, Table 2 depicts the symbols along with their description.

3.1. Network

In this study, we consider a multi-server, multi-user MEC environment with several small cells, S_c as shown in Fig. 1. Each S_c represents a network coverage over a limited area. The small cells enable low-latency edge computing capability and enhanced network coverage. Each S_c comprises m number of user equipment $U = \{ue_1, ue_2, ue_3, \ldots, ue_m\}$ and 'n' number of edge servers $E = \{es_1, es_2, es_3, \ldots, es_n\}$ such that $\{es_n, ue_m\} \in S_c$. Each edge server has three parameters such that $es = (\mu_n, Q_n, M_n)$. Here, μ_n represents the computational capability of an edge server, Q_n represents the resource (number of cores) of an edge server, and M_n is the memory of the edge server. The computational capability of the edge server is measured in FLOPS, which represent the computations performed by the processor per second and are relevant for the computation of the tasks. The unit to measure Q_n is COUNT, which is used to compute the task and is directly related to the ability of the edge server to handle multiple tasks.

3.2. Task

Each task $i \in \theta_i$ generated by user m at time t has four parameters, such that: $\theta_i, m, (t) = \langle I_i, C_i, R_i, T_i \rangle$ where, I_i is the input size of the task, C_i is the number of CPU cycles required for the computation of the task and R_i represents the resource (number of cores required) to compute the task and T_i represents the time requirement of the task. Each, ue_m within a given time frame $t \in T$, has the potential to generate a set of tasks θ_i. We assume that the task priorities are already set and is out of the scope of this study. Each edge server has a M/M/1 queue that stores the offloaded tasks. For an offloading decision ϕ_i, 0 indicates processing the task locally, while 1 indicates the task to be offloaded to an appropriate edge server es_n, such that the :

$$\phi_{i,m(t)} \in \{0, 1\}, \quad \begin{cases} \phi_i = 0 & \rightarrow \text{Local Execution} \\ \phi_i = 1 & \rightarrow \text{Server Execution} \end{cases} \tag{1}$$

The task can only be offloaded if the edge server has sufficient available resources, specifically the required number of CPU cores such that:

$$R_i < Q_n \quad \forall(i, m, n, t) \tag{2}$$

This ensures that the offloading decision respects the resource constraints and avoids overloading the server. It enables effective resource matching between tasks and edge servers. Hence, the tasks are not only offloaded however also placed on the most suitable resources for reliable and timely completion.

3.3. Task latency

3.3.1. Local computation

If the task θ_i of user m is processed on the local device, the delay for the local device to process the task can be obtained by:

$$L^{\text{local}}_{i,m}(t) = \frac{C_i}{f_m} \tag{3}$$

where C_i represents the number of CPU cycles required to compute the task and f_m denotes the computing resources of the local device m.

3.3.2. Edge computation

The offloading scheme significantly impacts the transmission latency, the computation latency, the propagation latency, and the queuing latency. Hence, we define the latency function $L_\theta = f(\theta)$ that calculates the total latency of the offloading scheme such as:

$$L^{\text{offload}}_{i,m,n}(t) = L^{\text{trans}}_{i,m,n}(t) + L^{\text{prop}}_{i,m,n} + L^{\text{queue}}_{i,n}(t) + L^{\text{comp}}_{i,n}(t) \tag{4}$$

The transmission latency is the delay to transmit the task. It depends upon the task size and the uplink rate. However, the downlink rate is negligible in this scenario, as the rate of returning the results is higher than the upload rate, so we neglect the downlink rate [15,46]. Hence, the task transmission latency depends on the input size of the task i and the task upload rate of the network channel between the user m and edge server n at time t.

$$L^{\text{trans}}_{i,m,n}(t) = \frac{I_i}{U_{i,m,n}(t)} \tag{5}$$

Using Shannon's theorem, we can write the equation for the task upload rate as:

$$U_{i,m,n}(t) = b \log_2(1 + SINR_{i,m,n}(t)) \tag{6}$$

where, b is the bandwidth and $SINR$ is the Signal-to-Interference-plus-Noise Ratio (SINR), which quantifies the quality of the communication link and is described as:

$$\text{SINR}_{i,m,n}(t) = \frac{P_{r,i,m,n}(t)}{P_i + N_0} \tag{7}$$

where P_r is power received, P_i is the total power of signals interfering with the desired transmission, and N_o is the thermal noise in the

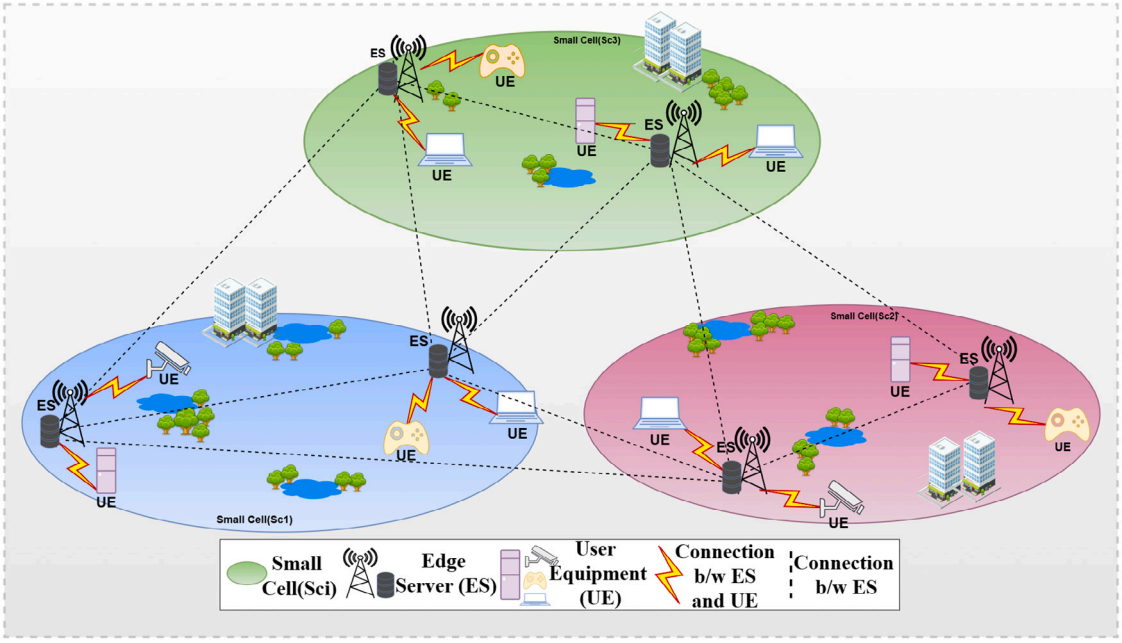


Fig. 1. MEC environment.

communication channel. However, the $\log_2(1+SINR)$ is a non-linear function, which is computationally expensive for the data transmission between *ue* and *es*, hence, we can linearize this function using first order Taylor expansion, such as for the function $f(SINR)$ we will take a reference point, such as $SINR_0$ and take the derivative. For computational efficiency in the online decision loop we apply a first-order Taylor expansion of $f(x) = \log_2(1+x)$ around a reference point $x = \text{SINR}_0$ such as:

$$f'(\text{SINR}_0) = \frac{1}{(1+\text{SINR}_0)\ln 2} \tag{8}$$

$$U_{i,m,n}(t) \approx b\left(\log_2(1+\text{SINR}_0) + \frac{1}{(1+\text{SINR}_0)\ln(2)}\left(\text{SINR}_{i,m,n}(t) - \text{SINR}_0\right)\right) \tag{9}$$

This linearization enables the faster updates for the offloading decision by simplifying the non-convex logarithm. The approximation accuracy is validated formally with an error bound in Appendix. SINR directly impacts the transmission latency and communication reliability. Higher SINR results in a lower task failure rate, enabling higher communication reliability. Furthermore, it is important to consider the signal attenuation, referred to as path loss that occurs as the signal propagates from the transmitter to the receiver. A higher path loss, due to increased distance, could lead to a drop in the SINR which contributes to unreliable communication links, reducing the efficiency of task offloading and increasing the risk of task failure. Now, assuming that the path loss is proportional to distance (d) hence:

$$P_{r,i,m,n}(t) = \frac{P_{i,m,n}(t)}{d_{m,n}^2} \tag{10}$$

where P_t represents the transmission power. Similarly, the time for the propagation of the signal over a distance d is calculated as:

$$L_{i,m,n}^{\text{prop}}(t) = \frac{d_{m,n}}{c} \quad \text{where} \quad c = 3\times10^8\,\text{m/s}\,(\text{speed of light}) \tag{11}$$

However, the tasks may experience queuing delays at the edge servers, which can be modeled using the M/M/1 queuing theory. Let $a_n(t)$ denote the task arrival rate at server n at time t, which is given by the total number of tasks offloaded to server n from all users within the time interval Δt:

$$a_n(t) = \frac{\sum_{m\in U}\sum_{i\in\theta_m}\phi_{i,m}(t)}{\Delta t} \tag{12}$$

Similarly, let $s_n(t)$ denote the service rate of server n, the rate at which the server can process tasks, calculated based on the server computational capability μ_n and the CPU cycles C_i required for task i:

$$s_n(t) = \frac{\mu_n}{C_i} \tag{13}$$

Then, the queuing delay experienced by task i at server $n$can be expressed as:

$$L_{\text{queue},i,n}(t) = \frac{a_n(t)}{s_n(t)\left(s_n(t) - a_n(t)\right)} \tag{14}$$

Finally, the computation latency can be described as the time taken by the server n to process task $i$and is given by:

$$L_{\text{comp},i,n}(t) = \frac{C_i}{\mu_n} \tag{15}$$

3.4. Energy consumption

3.4.1. Local computation

The energy consumption produced by processing tasks locally can be obtained by:

$$E_{i,m}^{\text{local}}(t) = C_i \cdot \eta_m \tag{16}$$

where η_m is the energy consumption per CPU cycle of the user device m. Note that different user devices exhibit different performance when processing user requests.

3.4.2. Edge computation

The energy consumption can be described as the energy required to offload the task and then to process it. Hence, the total energy consumption for offloading the task is the sum of the transmission energy (energy required to offload the task) and the computation energy (energy required for the computation of the task):

$$E_{i,m,n}^{\text{offload}}(t) = E_{i,m,n}^{\text{trans}}(t) + E_{i,n}^{\text{comp}}(t) \tag{17}$$

The computational energy depends upon the CPU cycle required and energy per CPU cycle, which varies depending on the efficiency of the device (such as for a local device it is 1×10^{-8} J/cycle for the edge server is 1×10^{-9} J/cycle. However, as each CPU instruction has different energy consumption depending on the instruction type and the device architecture, and additionally, when the system is under load, the operating system itself consumes energy to manage tasks and interact with the hardware components. Hence, the overall energy consumption of the computation process is:

$$E_{i,n}^{\text{comp}}(t) = C_i \cdot \eta_n + E_n^{\text{base}} + E_n^{\text{os}} \tag{18}$$

Where E_n^{base} is the base energy consumption of the CPU, which is the energy consumed when the CPU is idle or not processing any tasks, and E_n^{os} is the additional energy required by the operating system to manage the task. However, for the loaded edge server, when the queuing delays significantly impact task processing, the computation energy is adjusted to increased energy per cycle due to high utilization. The transmission energy can be described as:

$$E_{\theta_i}^{\text{trans}} = P_t \cdot L_{i,m,n}^{\text{trans}}(t) \tag{19}$$

3.5. Task reliability

In order to analyze whether the task offloading is reliable depends upon the communication reliability and the computation reliability [45]. The communication reliability ensures the transmission of the tasks within the given time frame, whereas the computation reliability refers to the accurate computation within the given time frame. As we consider the communication reliability and the computation reliability based on the failure probabilities [47]. The failure cases are normalized to a Poisson distribution derived from the failure rate during the communication of the task and the failure rate during the computation of the task. The communication reliability of the task can be described as:

$$Re_{i,m,n}^{\text{comm}}(t) = e^{-\lambda \cdot L_{i,m,n}^{\text{trans}}(t)} \tag{20}$$

Where λ represents the failure rate during the transmission of the task and is an exponential decay. Similarly, the computation reliability of the task can be described as:

$$Re_{i,n}^{\text{comp}}(t) = e^{-\epsilon \cdot L_{i,n}^{\text{comp}}(t)} \tag{21}$$

Where ϵ represents the failure rate during the computation of the tasks. Hence, the total reliability of the task π can be described as the product of the communication and computation reliability of the tasks. If the value is 1 the system is reliable; however value 0 indicates an unreliable system [11]:

$$Re_{\text{offload}}(t) = \sum_{i \in \theta} \left(Re_{i,m,n}^{\text{comm}}(t) \cdot \prod_{n \in E} Re_{i,n}^{\text{comp}}(t) \right) \tag{22}$$

Substituting the values of the Eqs. (20) and (21) in (22), we get:

$$Re_{i,m,n}^{\text{offload}}(t) = e^{-\lambda \cdot L_{i,m,n}^{\text{trans}}(t) - \epsilon \cdot L_{i,n}^{\text{comp}}(t)} \tag{23}$$

For the task executed locally at time t the reliability is calculated as an exponential decay with the execution time, longer local execution increases the failure probability, hence:

$$Re_{i,m}^{\text{local}}(t) = \mathrm{e}^{-\epsilon\, L_{i,m}^{\text{local}}(t)} \tag{24}$$

3.6. Problem model

Based on the Eqs. (3) and (4) the total latency can be written as:

$$L_{\text{total}}(t) = \sum_{m \in \mathcal{U}} \sum_{i \in \theta} \sum_{n \in \mathcal{E}} \Big((1 - \phi_{i,m}(t)) L_{i,m}^{\text{local}}(t) + \phi_{i,m}(t) L_{i,m,n}^{\text{offload}}(t) \Big) \tag{25}$$

Similarly, based on Eqs. (16) and (17), the energy consumption for local computation and transmitting tasks to the edge server can be expressed as:

$$E_{\text{total}}(t) = \sum_{m \in \mathcal{U}} \sum_{i \in \theta} \sum_{n \in \mathcal{E}} \Big((1 - \phi_{i,m}(t)) E_{i,m}^{\text{local}}(t) + \phi_{i,m}(t) E_{i,m,n}^{\text{offload}}(t) \Big) \tag{26}$$

Based on Eqs. (23) and (24) the total reliability can be described as:

$$Re_{\text{total}}(t) = \sum_{m \in \mathcal{U}} \sum_{i \in \theta} \sum_{n \in \mathcal{E}} \Big((1 - \phi_{i,m}(t)) Re_{i,m}^{\text{local}}(t) + \phi_{i,m}(t) Re_{i,m,n}^{\text{offload}}(t) \Big) \tag{27}$$

In a binary offloading, when the number of *ue* increases, the computational load also increases exponentially, and results in high energy consumption, high delay and an unreliable task offloading. The intractability stems from the combinatorial explosion in our problem formulation. Our objective function optimizes over three types of decision variables, binary offloading decisions Eq. (33), discrete server selection $n_{i,m}(t)$ for each offloaded task, and continuous power allocation $P_{i,m,n}(t)$ Eq. (34). With m users each generating tasks from set θ, we have $(u \cdot \theta)$ binary offloading decisions, and for each offloaded task, we must select from n available edge servers. This creates a decision space of size $O(2^{(u \cdot \theta)} \times n^i)$ where i represents the number of offloaded tasks. The multi-objective nature with coupled constraints (Eqs. (29)–(32)) further complicates the solution methods. Hence, this can be termed as a NP-hard problem [27,38]. We optimize the task offloading strategy by minimizing the time delay and energy consumption and maximizing the reliability in terms of communication and computation.

$$\min_{\phi_{i,m}(t), n_{i,m}(t), P_{i,m,n}(t)} \sum_{t \in T} \Big[\alpha\, L_{\text{total}}(t) + \beta\, E_{\text{total}}(t) - \gamma\, Re_{\text{total}}(t) \Big] \tag{28}$$

Subject to:

$$R_i(t) \le Q_n, \quad \forall i \in \theta,\ \forall n \in \mathcal{E},\ \forall t \in \mathcal{T} \tag{29}$$

$$L_{\text{total}}(t) \le L_{\max}, \quad \forall t \in \mathcal{T} \tag{30}$$

$$E_{\text{total}}(t) \le E_{\max}, \quad \forall t \in \mathcal{T} \tag{31}$$

$$0 < Re_{\text{total}}(t) \le 1, \quad \forall t \in \mathcal{T} \tag{32}$$

$$\phi_{i,m}(t) \in \{0, 1\}, \quad \forall i \in \theta,\ \forall m \in \mathcal{U},\ \forall t \in \mathcal{T} \tag{33}$$

$$P_{\min} \le P_{i,m,n}(t) \le P_{\max}, \quad \forall i \in \theta,\ \forall m \in \mathcal{U},\ \forall n \in \mathcal{E},\ \forall t \in \mathcal{T} \tag{34}$$

The objective goal is minimizing the energy consumption and latency and maximizing the reliability. The objective function is the sum of all the latencies, energy consumption, and reliability of all the tasks. The constraint (29) represents the resource constraint, as the resource to calculate the task must be less than the resource of the edge server; the constraint (30) indicates that the total latency must be less than the threshold latency. (31) presents the energy consumption constraint, which must be less than the total energy consumption, the constraint (32) presents the reliability value as the value between 0 and 1. (33) is the offloading decision constraint, which represents the binarization of the task offloading. Finally (34) transmission power used for offloading task i from user m to server n at time t.

4. RELTO algorithm

This paper proposes a DRL-based multiobjective optimization algorithm, that unifies a dual-attention transformer structure with the PPO algorithm to adaptively manage multi-objective task offloading in MEC scenario. To address the intractability of problem formulation, RELTO leverages several mechanisms such as, the feature attention module (Section 4.1) compresses the high-dimensional system state into a manageable global context vector h_t. Similarly, rather than exploring all possible combinations of offloading and server selection decisions, RELTO uses action masking (Section 4.2) to constrain exploration to feasible regions defined by the resource constraint Eq. (29), and the PPO framework ensures stable policy updates through clipped objectives Eq. (61). The reward weighting attention (Section 4.3) dynamically balances objectives. We model the environment for reinforcement learning by defining structured state, action, and reward spaces as follows:

- State: The state space $s_t$such that $(s_t \in S)$ can be described as the tuple containing the information of the task θ_i, generated by the user equipment m, the edge server E_n and the distance between the user m and edge server n such as:

$$S = \left\{ s_t \;\middle|\; s_t = \{\theta_i(t), m, n, d_{m,n}\},\ \forall i \in \theta,\ m \in \mathcal{U},\ n \in \mathcal{E} \right\} \tag{35}$$

- Action: In this model, the DRL-agent will take the action a_t such that $(a_t \in A)$ based on the current state s_t. The goal of the agent is to construct an optimal decision in order to maximize the reliability in terms of communication and computation and minimize the overall delay with less energy consumption by capturing the discrete as well as continuous action spaces $a_t = \left(\{ a^{\text{disc}}_{i,m}(t), a^{\text{cont}}_{i,m,n}(t) \} \right)$. The action a at time t can be described as:

$$A = \left\{ a_t = \left(\phi_{i,m}(t), n_{i,m}, P_{i,m,n}(t) \right) \;\middle|\; i \in \theta,\ m \in \mathcal{U},\ n \in \mathcal{E},\ t \in \mathcal{T} \right\} \tag{36}$$

  where the offloading decision $\phi_{i,m}(t)$ and the server selection $n \in E$ represents the discrete action. Moreover, choosing optimal transmission power $P_{i,m,n}$ for offloading based on network conditions represents the continuous action space.
- Reward: The reward signal reflects a scalarized objective function containing the multi-objectives, where the objective is to minimize the total energy consumption with the total latency and maximize the total reliability. The raw metrics (L_t, E_t, Re_t) are obtained after executing an action and are normalized and subsequently combined with adaptive scalar weights:

$$R_{t,\text{adaptive}} = -w^t_1 \cdot L^{\text{norm}}_t - w^t_2 \cdot E^{\text{norm}}_t + w^t_3 \cdot R^{\text{norm}}_{et} \tag{37}$$

  These weights are dynamically learned through the reward weighting attention, allowing the agent to prioritize objectives based on real-time context rather than relying on static reward coefficients.

RELTO employs hybrid action spaces and sophisticated reward shaping to ensure convergence to an optimal solution as compared to conventional DRL. The system architecture is divided into four functionally distinct modules, feature attention module, PPO policy network (Actor), PPO value network (Critic) and reward weighting attention module as shown in Fig. 2. The design ensures that raw environmental states are effectively abstracted, contextually weighted for goal prioritization, and fed into PPO actor-critic system for decision-making. This integrated architecture enables the agent to perform adaptive, context-aware offloading strategies in a dynamic MEC environment.

4.1. Observation and feature attention

The first block of the proposed framework is the feature attention. At time t, the agent receives the state observation$o_t \in \mathbb{R}^{N \times D_{\text{obs}}}$, where N is the number of entities and D_{obs} is the dimension of features per entity, encapsulating the current status of users, computational tasks, edge servers, and the wireless channel environment. This observation is carefully structured to retain interdependencies between multiple heterogeneous system components, providing a rich feature space for the attention mechanism to capture meaningful interactions. This feature attention module is responsible for generating a global, context-rich latent vector from raw observations. It is implemented using a deep transformer encoder comprising multiple identical layers, each consisting of Multi-Head Self-Attention (MHSA), residual skip connections, and position-wise feedforward networks. Each input entity vector is first linearly embedded [48]:

$$E_t = O_t W_E + b_E \tag{38}$$

where $W_E \in \mathbb{R}^{D_{\text{obs}} \times D_{\text{model}}}$ and $b_E \in \mathbb{R}^{D_{\text{model}}}$ are learnable embedding parameters. This embedded input passes through a multi-layer Transformer encoder composed of L identical layers.

Each Transformer layer computes attention using the scaled dot-product self-attention mechanism [49]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} \right) V \tag{39}$$

where Q, K, V represented as the query, key, and value matrices respectively. d_k is the dimension of the keys, and QK^T is the matrix transpose operator. The value vectors in matrix V are weighted using the weights resulting from the softmax operation as:

$$Q = E_t W^Q_i, \quad K = E_t W^K_i, \quad V = E_t W^V_i \tag{40}$$

for each attention head $i \in \{1, \ldots, H\}$. The outputs of all heads are concatenated and linearly transformed, followed by a residual connection and layer normalization:

$$\text{MHSA}(E_t) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H) W^O \tag{41}$$

$$X' = \text{LayerNorm}(X + \text{MHSA}(X)) \tag{42}$$

Each token is then passed through a position-wise feedforward network and another residual connection and normalization such as:

$$\text{FFN}(X) = \text{ReLU}(X W_1 + b_1) W_2 + b_2 \tag{43}$$

$$X'' = \text{LayerNorm}(X' + \text{FFN}(X')) \tag{44}$$

The entire Transformer block is applied L times sequentially, and the final token-wise outputs are pooled to form the global context vector

$$h_t = \text{Pool}(X_L) \tag{45}$$

This global vector encodes the entire system state in a compact, latent representation suitable for downstream decision-making.

4.2. Policy and value function networks (actor-critic PPO)

The global context vector h_t, extracted using the feature attention, is shared as input to both the actor and critic networks within the PPO framework. This design enables the policy and value functions to operate on a common, contextually rich representation of the environment.

Actor Network (Hybrid Action Design): The actor network is structured to accommodate both discrete and continuous action spaces. It is composed of a shared multi-layer perceptron (MLP) backbone followed by two distinct output heads. The shared backbone includes fully connected layers with ReLU activation and a dropout layer to mitigate overfitting.

$$z_t = \text{MLP}_{\text{shared}}(h_t) \tag{46}$$

- Discrete Action Head: This head outputs logits corresponding to multiple discrete decisions. These decisions include, whether to offload a task (binary decision) and which server to assign (categorical). To prevent the policy from exploring infeasible actions, we apply action masking to the logits [50]. The mask, generated at each time step as a binary mask (1 for feasible actions and 0 for invalid actions, such as if a server has insufficient CPU capacity for a task, the mask value for offloading to that server is set to 0) based on the current system state and resource availability, sets the logits of invalid actions to a very low negative value $(-\infty)$ effectively making their selection probability zero after the softmax operation. This method significantly improves the training stability and sample efficiency by constraining the agent exploration to a feasible subspace of the action space. After masking, each decision dimension is handled by a separate softmax layer:

$$\pi^{\text{disc}}_i(a^i_t \mid h_t) = \text{softmax}(Mask(z^{\text{disc},i}_t)), \quad \forall i \in \{\text{offload}, \text{server}\} \tag{47}$$

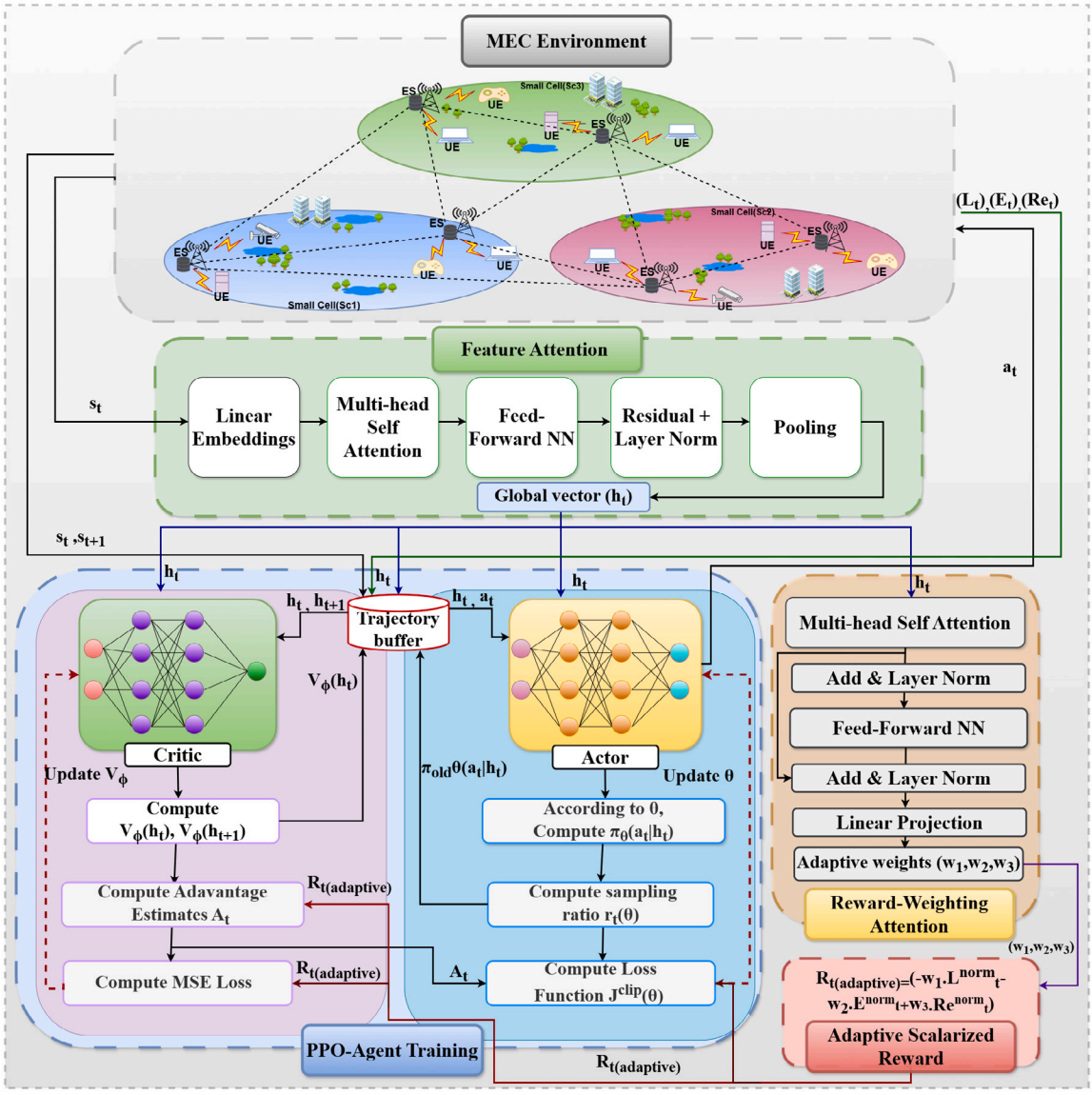


Fig. 2. The proposed RELTO architecture for the task offloading in MEC environment.

- Continuous Action Head: Designed for fine-grained control specifically for transmission power adjustment, this head outputs the parameters of a Gaussian distribution:

$$\mu_t^{\text{cont}}, \log \sigma_t^{\text{cont}} = \text{MLP}_{\text{cont}}(z_t) \tag{48}$$

$$a_t^{\text{cont}} \sim \mathcal{N}(\mu_t^{\text{cont}}, (\sigma_t^{\text{cont}})^2) \tag{49}$$

The final action a_t is a hybrid vector formed by sampling from the discrete and continuous heads. This design allows the agent to capture both logical decisions and continuous control in a unified policy:

$$\pi(a_t \mid h_t) = \prod_i \pi_i^{\text{disc}}(a_t^i \mid h_t) \cdot \mathcal{N}(a_t^{\text{cont}} \mid \mu_t^{\text{cont}}, \sigma_t^{\text{cont}^2}) \tag{50}$$

Critic Network: The critic network also takes $\mathbf{h}_t$ as input and outputs a scalar estimate of the value function:

$$V(h_t) = \text{MLP}_{\text{critic}}(h_t) \in \mathbb{R} \tag{51}$$

This value is used to calculate advantage estimates for PPO loss computations, leveraging the shared perception from h_t. Both MLPs consist of fully connected layers, ReLU activations, and optional dropout for regularization. The shared input h_t ensures both actor and critic are grounded in the same contextual representation.

4.3. Reward weighting attention

The primary role of this attention block is to make the DRL-agent learning process more intelligent and adaptable by allowing it to dynamically prioritize the performance objectives based on the current context. This block generates the scalar coefficients (w_1^r, w_2^r, w_3^r) that will be used to combine the raw performance metrics (L_t, E_t, Re_t) into a single, comprehensive reward signal $R_{t,\text{adaptive}}$. Unlike traditional methods, where these weights are static, this module learns to set them adaptively. The vector h_t is passed through a shallow Transformer encoder, tailored to transform high-level context into reward priorities. Since the input here is already a condensed "global vector", fewer layers are sufficient here compared to the main feature extractor. The h_t vector is reshaped to be compatible with the transformer encoder input format as, (batch_size, sequence_length, embedding_dimension). Here, we treat h_t as a sequence of length 1 token:

$$x_{\text{rw}} = h_t.\text{unsqueeze}(1) \quad (\text{shape: } (\text{batch_size}, 1, D_{\text{features}})) \tag{52}$$

The transformer encoder layer applies self-attention and a Feed-Forward Network (FFN) to x_{rw}. Although it is single token, the self-attention mechanism within the layer still allows for complex transformations

and feature mixing relevant to deriving the weights:

$$x'_{\text{rw}} = \text{TransformerEncoderLayer}(x_{\text{rw}}) \tag{53}$$

The output x'_{rw} is flattened back to (batch_size, D_{features}) and passed through a linear layer to produce raw weight values:

$$\text{weights}_{\text{raw}} = \text{LinearProjection}(x'_{\text{rw}}.\text{squeeze}(1)) \tag{54}$$

Afterwards, a non-negative activation function (softplus) is applied to ensure the weights are positive, which is crucial for meaningful linear scalarization. A small epsilon is added for numerical stability:

$$w_i = \text{Softplus}(\text{weights}_{\text{raw}}[i]) + \epsilon \quad \text{for} \quad i \in \{1, \ldots, N_{\text{weights}N=3}\} \tag{55}$$

The output of this block is a tuple of adaptive scalar weights:

$$(w_1^t, w_2^t, w_3^t) \in \mathbb{R}^{N_{\text{weights}}} \tag{56}$$

4.4. Adaptive reward scalarization ($R_{t,adaptive}$)

In this module, the outputs from the reward weighting attention are applied to form the learning signal for the PPO. Before combining, the raw objective values $(L_t), (E_t), (Re_t)$, are normalized using Z-score standardization to ensure the scale invariance. This prevents one objective from dominating the others, which is essential for stable training. We implement the following normalization procedure to standardize each raw metric:

$$L_t^{\text{norm}} = \frac{L_t - \mu_L}{\sigma_L} \tag{57}$$

$$E_t^{\text{norm}} = \frac{E_t - \mu_E}{\sigma_E} \tag{58}$$

$$R_{et}^{\text{norm}} = \frac{R_{et} - \mu_R}{\sigma_R} \tag{59}$$

Here, (μ_i) and σ_i are the running mean and standard deviation of each reward component. These values are computed and updated after each episode using a running average method to account for the non-stationary nature of the environment and ensure the normalization is consistent across the entire training process. The normalized metrics are then combined with the adaptive weights w_1^t, w_2^t, w_3^t obtained from the reward weighting attention for specific timestep t.

$$R_{t,\text{adaptive}} = -w_1^t \cdot L_t^{\text{norm}} - w_2^t \cdot E_t^{\text{norm}} + w_3^t \cdot R_{et}^{\text{norm}} \tag{60}$$

The negative signs indicate that latency and energy are objectives to be minimized (penalties), while reliability is an objective to be maximized (reward). The weights w_1^t, w_2^t, w_3^t are always positive due to the Softplus activation in the reward weighting attention while Z-score standardization ensures the metrics being weighted are not on vastly different scales which is essential for stable training.

4.5. PPO objective function

The PPO aims to maximize the following clipped surrogate objective function:

$$L_{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{61}$$

where: $\mathbb{E}_t$ denotes the empirical average over a batch of samples. $r_t(\theta) = \frac{\pi_{\theta_{\text{old}}}(a_t|s_t)}{\pi_\theta(a_t|s_t)}$ is the ratio of the new policy probability to the old policy probability. $\hat{A}_t$ is the Generalized Advantage Estimate (GAE) at timestep t. Crucially, $\hat{A}_t$ is computed using the adaptively scalarized reward $R_{t,\text{adaptive}}$ such as:

$$\hat{A}_t = R_{t,\text{adaptive}} + \gamma\lambda\hat{A}_{t+1} + \gamma V(h_{t+1}) - V(h_t) \tag{62}$$

For simplification $\hat{A}_t$ can be described as:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t-1}\delta_{T-1} \tag{63}$$

where $\delta_t = R_{t,\text{adaptive}} + \gamma V(h_{t+1}) - V(h_t)$ is the Temporal Difference (TD) error using the adaptive reward. Additionally, PPO optimizes a value function loss (L_{VF}) and an entropy bonus (S) to encourage exploration:

$$L_{\text{VF}}(\phi) = \mathbb{E}_t \left[\left(V_\phi(h_t) - V_t^{\text{target}} \right)^2 \right] \tag{64}$$

where V_t^{target} is the estimated return using $R_{t,\text{adaptive}}$. The entropy of the policy distribution is:

$$S[\pi_\theta] = \mathbb{E}_t \left[H(\pi_\theta(a_t \mid h_t)) \right] \tag{65}$$

Hence, the final objective for optimization is:

$$L(\theta, \phi) = L_{\text{CLIP}}(\theta) - c_1 L_{\text{VF}}(\phi) + c_2 S[\pi_\theta] \tag{66}$$

where c_1, c_2 are coefficients for the value function loss and entropy regularization, respectively.

4.6. Training process

The training process involves an iterative loop of data collection and model optimization as described in algorithm 1. The PPO agent interacts with the MEC environment for a predefined number of steps, known as a "rollout". This is a standard procedure in on-policy reinforcement learning algorithms. For each time step t of this interaction

- The current state of the MEC environment is captured as observations o_t and fed into the feature attention,to generate a context-rich "global vector" h_t. This h_t is a compressed, salient representation of the entire MEC system state. The global vector h_t is then fed into the PPO actor network, which outputs the action a_t, the PPO critic network, which estimates the value of the current state, $V(h_t)$ and the Reward Weighting Attention module to dynamically calculate the scalar coefficients (w_1^t, w_2^t, w_3^t) as described in algorithm 2. In response to action a_t, the environment returns raw performance metrics such as (L_t),(E_t) and (Re_t). All these values (o_t, h_t, a_t, $V(h_t)$, w_1^t, w_2^t, w_3^t, L_t, E_t, Re_t) are stored in a "rollout buffer". This buffer accumulates data over the fixed number of timesteps in a rollout. After the entire rollout is collected, for every timestep t within that rollout, the stored raw objectives (L_t, E_t, Re_t) are normalized using the running statistics for a specific timestep and then combined with their corresponding stored adaptive weights (w_1^t, w_2^t, w_3^t) to compute the final adaptive reward, $R_{t,\text{adaptive}}$. This means the reward for a given timestep is determined by the weights that were predicted at that very timestep, based on the state at that time.
- Loss Computation: Using the $R_{t,\text{adaptive}}$ values, advantage estimates $\hat{A}_t$ and V_t^{target} are computed. The $\hat{A}_t$ measure of how much better an action was compared to the average expected outcome from that state. The value targets V_t^{target} are the true (or estimated true) cumulative returns from a state, used to train the critic network. Afterwards, the $\hat{A}_t$ and V_t^{target} values are used to calculate the specific loss components such as L_{CLIP} (PPO clipped policy loss, which encourages policy updates while preventing them from being too large), L_{VF} (the value function loss, which trains the critic to accurately predict state values), and $S[\pi_\theta]$ (an entropy bonus, which encourages exploration by penalizing deterministic policies).
- Parameter Update: The total loss $L(\theta, \phi)$ is backpropagated through the entire network. This is the core of neural network training, where gradients are computed and propagated backward through the network layers. An Adam optimizer updates the parameters of all four neural network components. This end-to-end training ensures the feature attention produces discriminative global vectors, the reward weighting attention assigns effective adaptive weights, the PPO actor maximizes adaptively weighted cumulative reward, and the PPO critic accurately predicts state values. This collective optimization makes the agent robust to varying environmental conditions and objective priorities.

Algorithm 1 RELTO: Reliability, Energy, and Latency Optimization for MEC Task Offloading

Input: Number of Tasks $\theta(i)$, Number of Users m, Number of Edge Servers n, and Network Parameters
Output: Optimized Policy π_θ
Initialize policy parameters θ, value function parameters ϕ, Feature Attention and Reward Weighting Attention parameters $\theta_{\text{feat}}, \theta_{\text{rw}}$
Set initial environment state s_0
while not converged **do**
  for each user $ue = 1$ to m **do**
    for each task $\theta(i)$ **do**
      Observe current state s_t {FeatureAttention Transformer Block}
      $E_t \leftarrow \text{LinearEmbedding}(s_t; W_{\text{emb}}, b_{\text{emb}})$
      for each layer l in transformer encoder **do**
        $Q, K, V \leftarrow$ project from E_t
        $\text{MHSA}_l \leftarrow \text{MultiHeadSelfAttention}(Q, K, V)$
        $\text{Norm1}_l \leftarrow \text{LayerNorm}(\text{MHSA}_l + \text{Input})$
        $\text{FFN}_l \leftarrow \text{FeedForward}(\text{Norm1}_l)$
        $\text{LayerOutput}_l \leftarrow \text{LayerNorm}(\text{FFN}_l + \text{Norm1}_l)$
      end for
      $h_t \leftarrow \text{GlobalPooling}(\text{LayerOutput}_L)$
      {Adaptive Reward Weighting}
      $(w_{1,t}, w_{2,t}, w_{3,t}) \leftarrow$ Reward Weighting Attention$(h_t; \theta_{\text{rw}})$ using Algorithm 2 {Policy Action Selection}
      $a_t \sim \pi_\theta(a_t \mid h_t)$
      Compute log probability $\log \pi_\theta^{\text{old}}(a_t \mid h_t)$
      if a_t indicates offloading **then**
        Select edge server $j \in \{1, \ldots, n\}$ from a_t
      end if{Reward Scalarization}
      Observe raw objcetives: L_t, E_t, Re_t from environment info and normalize
      Compute Adaptive Reward using (60)
      Store $(s_t, h_t, a_t, \log \pi_\theta^{\text{old}}, V(h_t), w_{1,t}, w_{2,t}, w_{3,t}, L_t, E_t, Re_t, s_{t+1}, R_{t,\text{adaptive}})$ in Trajectory Buffer
    end for
  end for
  Compute the Advantage estimates A_t using Eq. (63)
  Update the policy using Eq. (66)
  Update Value Function using Eq. (64)
end while

Algorithm 2 Adaptive Reward Weighting using Attention Transformer

Input: Global state representation h_t (from FeatureAttention), TrajectoryBuffer
Output: Adaptive weights $(w_{1,t}, w_{2,t}, w_{3,t})$
Initialize: Parameters of Reward Weighting Attention θ_{rw}
for each timestep t in rollout **do**
  Input: Global context vector h_t
  Predict Adaptive Weights:
  $x_{\text{rw}} \leftarrow h_t.\text{unsqueeze}(1)$
  for each layer l in Reward Weigthing Attention **do**
    Apply MHSA: $q, k, v \leftarrow W'_Q x, W'_K x, W'_V x$
    $z_l \leftarrow \text{MHSA}(q, k, v)$
    $z_l \leftarrow \text{LayerNorm}(z_l + x)$
    $z_l \leftarrow \text{FFN}(z_l)$
    $x \leftarrow \text{LayerNorm}(z_l + z_l)$
  end for
  Output Weights: $(w_1^r, w_2^r, w_3^r) \leftarrow \text{Linear}(x)$
  Activate: $w_{i,t} \leftarrow \text{Softplus}(w_i^r) + \epsilon \quad \{\text{for } i = 1, 2, 3\}$
  Store: $(w_{1,t}, w_{2,t}, w_{3,t})$ in TrajectoryBuffer
end for

4.7. Complexity analysis

The RELTO framework overall time complexity for a single training step is determined by the sequence of operations such as, a forward pass through the Feature Attention network, a forward pass through the Reward Weighting Attention module, and the forward passes through the final linear layers of the actor and critic networks, followed by backward passes for gradient computation. Let N be the total number of entities in the system (tasks, user equipment, and edge servers), d_{model} be the feature vector dimension, L_{fa} be the number of layers in the Feature Attention network and d_k be the dimension of a single head, where $d_k = \frac{d_{\text{model}}}{h}$ (h is the number of heads). The time complexity of a single attention layer is determined by the $Q \cdot K^T$ multiplication, which scales as, $O(N \cdot d_k \cdot N)$. This simplifies to $O(N^2 \cdot d_k)$. Since the Feature Attention network has L_{fa} layers, its total time complexity for a single forward pass is, $O(L_{\text{fa}} \cdot N^2 \cdot d_k)$. The Reward Weighting Attention module is designed to process a single global vector, allowing it to generate adaptive reward weights efficiently. This component operates on a constant-sized input, and its computational complexity is therefore independent of the number of entities, expressed as, $O(1)$. Similarly, the final linear layers of the actor and critic networks that process the output of the attention mechanism also have a complexity that does not scale with the number of entities, expressed as, $O(1)$. The complexity of these subsequent operations is significantly less than that of the Feature Attention network and constant. Therefore, the total time complexity for one forward-and-backward pass is dominated by the Feature Attention network The total time complexity for the entire training process is a function of the total number of training steps T multiplied by the complexity of each step:

$$\text{Total Time Complexity}_{\text{RELTO}} = T \cdot O(L_{\text{fa}} \cdot N^2 \cdot d_k) = O(T \cdot N^2) \tag{67}$$

Hence, the computational intensity of the RELTO arises primarily from the modeling capabilities of the Transformer. While this architecture entails higher computational costs than traditional methods, its ability to effectively capture complex temporal and spatial dependencies greatly enhances the accuracy and efficiency of decision-making.

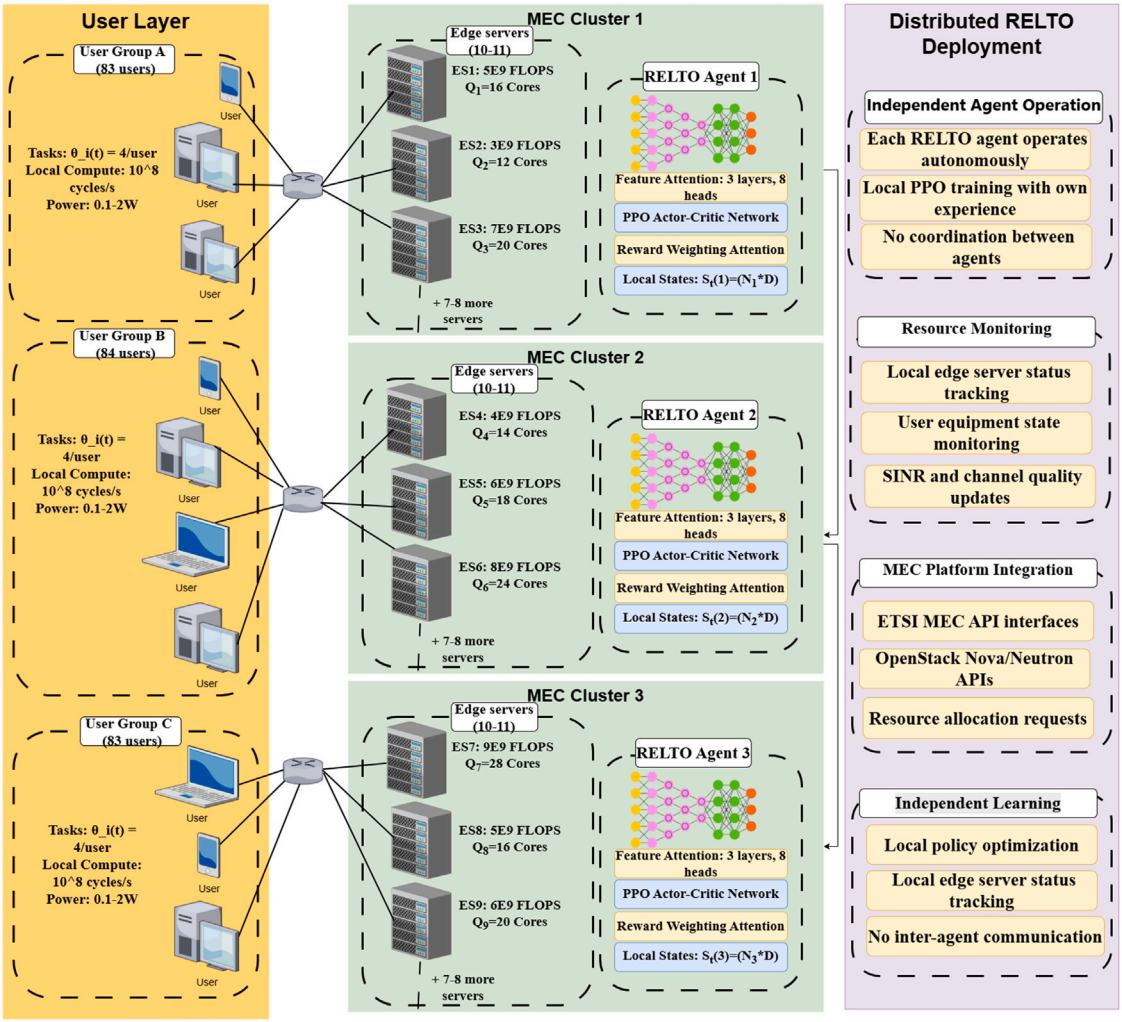


Fig. 3. System deployment architecture.

4.8. Deployment architecture and feasibility analysis

4.8.1. Deployment architecture

This section presented the discussion on the deployment architecture of RELTO. In this architecture, RELTO can be deployed at regional edge controllers, which coordinate multiple base stations within a localized area. This structure allows for proximity-based decision-making and minimizing latency The distributed RELTO system architecture represents a multi-cluster deployment where independent RELTO agents operate across geographically distributed MEC infrastructure without coordination between agents. The edge nodes perform millisecond-level offloading decisions using these pre-trained RELTO models. By placing the inference logic at the edge, the system avoids the latency associated with sending every decision to the cloud, enabling real-time responsiveness for mobile applications.as shown in Fig. 3.

4.8.2. Feasibility analysis

The feasibility of this architecture is supported by an analysis of its hardware, network, and scalability requirements.

- Hardware Requirements: Our analysis indicates that standard edge servers (such as, Intel Xeon processors with 32 GB RAM) are more than capable of handling RELTO inference. With our complexity analysis showing RELTO has a potential of just 2.85 ms decision latency. This minimal overhead makes the system practical for a wide range of real-world scenarios.
- Network Integration: RELTO is designed to be compatible with existing MEC platforms. It can be integrated through standard REST APIs, allowing it to seamlessly interface with orchestration tools such as, OpenStack and Kubernetes.
- Scalability: The hierarchical, distributed deployment model ensures high scalability. Based on our complexity analysis, a single edge cluster can effectively support anywhere from 100 to 1,000 users, making it suitable for high-density environments.

5. Experiment analysis and discussion

5.1. Experimental setup

In our simulation, we modeled a mobile edge computing environment with a number of users ranging from 50 to 250 [36], each generating four tasks with varying computation, time, and memory requirements. The Local computational capabilities of 10^8 cycle/s [36] consume 1×10^{-8} J/cycle energy for the task execution. Edge servers ranging from 4 to 32 [22] were deployed at fixed locations within

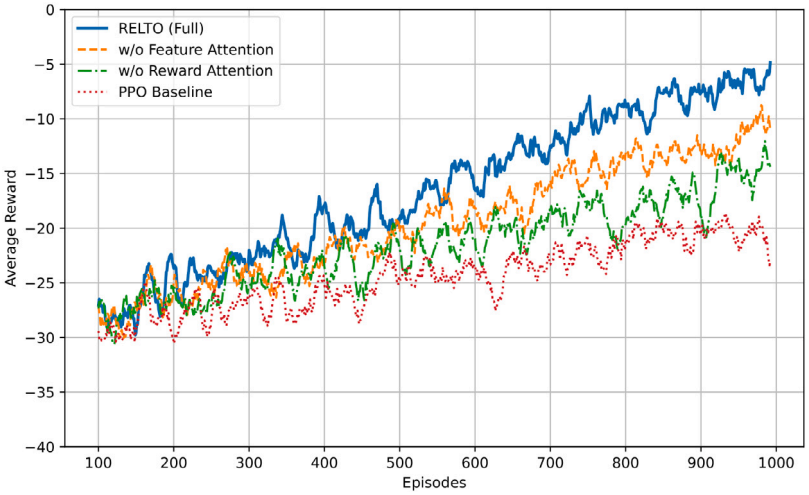


Fig. 4. Convergence comparison of RELTO variants.

Table 3
Hyperparameters and their Values.

Hyperparameter	Value
Learning Rate	0.0001
Discount Factor	0.99
Batch size	128
Clip Range	0.2
Entropy Coefficient	0.01
GAE (Generalized Advantage Estimation)	0.95
Number of Feature Attention Layers	3
Number of Feature Attention Heads	8
Number of Feature Attention embeddings	128
Number of Reward Weighting Attention Layers	1
Number of Reward Weighting Attention Heads	4
Number of Reward Weighting Attention embeddings	128
Dropout rate	0.1

a 5000 × 5000 m simulation area, with coverage areas defined by transmit power and the bandwidth ranging from 20 to 500 MHz [15,36], with the computational capability ranging from 1e7 to 1e10 cycles/s [22] and energy consumption of 1×10^{-9} J/cycle. Communication links were modeled using SINR-based channel conditions, incorporating path loss and thermal noise. The service utilization rates (ρ) for edge servers ranged from 0.8 to 0.95, with average service rates (s) distributed normally around $s = 0.5$ tasks/ms. Task arrival rates (a) were calculated as $a = \rho s$, ensuring realistic load conditions. Each service request required 10M CPU cycles, with tasks generated at intervals following uniform distributions. Task failure rate during transmission (λ) ranges from 0.02 to 0.04, 0.06, and 0.08. Similarly, the task failure rate during the computation (ϵ) ranges from 0.02 to 0.04, 0.06, and 0.08 [36]. Each service request required 10M CPU cycles, with tasks. Adam optimizer is utilized with the learning rate of 0.0001 [22,48] and a batch size of 128 to provide stable gradient estimates while preserving sample efficiency. The activation function of the MLP layers is ReLU. Further details are provided in the Table 3. We conduct the simulation on the on NVIDIA RTX 3080 GPU with Windows platform and built a custom MEC environment using Python 3.6 and Torch 1.9.1 based on the above setup.

5.2. Performance evaluation

In this section, we demonstrate the quantitative gains of our proposed architectural components. We trained several variants of the RELTO framework by systematically modifying key modules and comparing their performance as shown in Fig. 4.

- RELTO (Full): The complete framework, including the dual-attention feature extractor and the adaptive reward weighting attention module. This model consistently achieves the highest average reward.
- w/o Feature Attention: A simplified model where the dual-attention transformer is replaced with a standard Multi-Layer Perceptron (MLP) for state representation. This model performance significantly lags behind the full RELTO model, proving the critical role of the attention mechanism in processing the high-dimensional, relational state space.
- w/o Reward Attention: A version of the model that uses a static, pre-defined reward function instead of the adaptive reward weighting attention. Its performance shows a clear drop compared to the full model, highlighting the importance of dynamically adjusting objective priorities based on real-time context.
- PPO Baseline: A basic PPO model with a simple MLP and a single-action space. This serves as our lowest performance benchmark and underscores the combined power of the attention architecture, and adaptive reward.

The convergence curves highlight the impact of the dual-attention architecture on stability and learning speed. The PPO baseline exhibits slow convergence, reflecting its inability to adapt under varying MEC conditions due to fixed scalarization of objectives. The w/o Feature Attention variant improves over PPO by leveraging adaptive reward weighting however, the limited representational capacity of the states constrains its ability to capture complex contexts, leading to moderate gains. Conversely, the w/o Reward Attention variant achieves richer state embeddings however, suffers from fixed reward trade-offs, causing slower convergence and weaker final performance than w/o Feature Attention. The full RELTO framework outperforms all variants, converging faster and achieving a substantially higher reward. Importantly, RELTO superiority emerges from the synergistic effect of feature attention (feature extraction) and reward attention (adaptive scalarization), which together enable robust, context-aware decision-making under dynamic MEC environments.

Similarly, Table 4 demonstrates the quantitative comparison. As expected, the PPO baseline achieves lowest inference latency (0.12 ms) due to its minimal parametrization, however this comes at the expense of the poor performance with and average reward of −18.81 and high convergence speed 1000+. Adding reward attention (w/o feature attention) increases the parameters leading to moderate rise in the inference latency however improves the average reward to −14.10. On the other hand, adding only the feature attention with a deeper transformer increases the parameters however doubles the inference

Table 4
Performance Comparison of RELTO variants.

Variant	Inference Latency	No. of parameters	Average Reward	Convergence Speed
PPO baseline	0.12 ms	66,893	−18.81	1000+
w/o Feature Attention	0.56 ms	304,583	−14.10	−900
w/o Reward Attention	1.07 ms	609,246	−16.02	−950
RELTO	2.58 ms	1,913,869	−4.68	−700

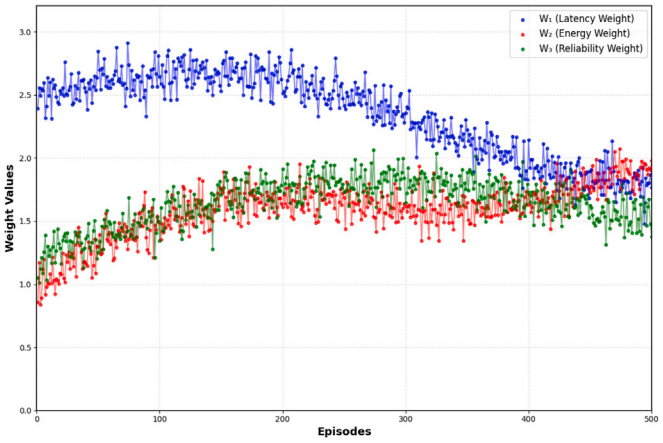


Fig. 5. Dynamic weights evolution over training episodes.

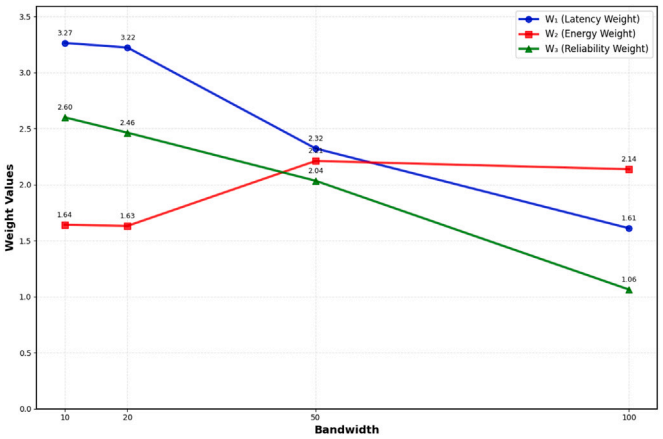


Fig. 6. Weight values for different bandwidth scenarios.

latency and with static rewards it performs significantly worse with an average reward of −16.02 reward and slow convergence. This directly demonstrates that the quality of the learning signal is more critical for robust policy learning than the raw complexity of the policy network itself. The RELTO model introduces high inference latency however, shows a significant performance gain with an average reward of −4.68 and fast convergence speed. This clearly illustrates the performance complexity trade-off. While dual-attention increases the computation demands, the adaptive and context aware nature of RELTO yields significantly higher robustness making the additional cost justifiable.

Fig. 5 illustrates the time-based behavior of the weights for Latency (w_1), Energy (w_2), and Reliability (w_3). The graph provides a macro-level view of the learning process, demonstrating that the weights are not static and the reward weighting attention module is actively learning to adapt the reward signal in a dynamic environment. In the initial Learning phase the agent initially places a high emphasis on minimizing latency ((w_1) is highest), which is a common strategy to ensure task completion. As the training progresses, the weights for energy (w_2) and reliability (w_3) gradually increase. After an initial learning phase, the weights enter a more stable and dynamic state. This demonstrates that the agent has successfully learned a robust, context-aware policy, with the weights continuously adjusting in response to subtle shifts in the MEC environment.

To provide a more granular and direct proof of the framework adaptivity, we analyze how the average weight values change in response to a critical environmental variable of network bandwidth as shown in the Fig. 6. Each data point on the graph represents the average value of the corresponding weight over an entire training episode where the network bandwidth is held constant at the specified level. This method allows us to demonstrate the stable, long-term policy learned by the agent for a given environment. In low-bandwidth scenarios (20 MHz), the latency weight (w_1) is at its highest. The agent correctly identifies latency as the primary bottleneck and prioritizes offloading decisions that mitigate it. As the bandwidth increases to 50 MHz, the latency weight (w_1) decreases. Simultaneously, the energy weight (w_2)increases. This demonstrates a sophisticated trade-off, the agent shifts its focus from minimizing latency to reducing energy consumption, as the network is no longer a major constraint on speed. Similarly, the reliability weight (w_3) is highest at low bandwidth. A poor channel is more prone to packet loss and transmission errors. The agent intelligently assigns a higher weight to reliability in at low bandwidth to ensure tasks are completed successfully despite the challenging conditions.

5.3. Comparative analysis

We select and implement seven baselines for the comparison, such as:

- DDPG (Deep Deterministic Policy Gradient) [7]: An off-policy model with continuous control and flattened state representation. It aims to optimize the task offloading decision and response speed through reasonable resource and computational allocation.
- A2C (Advantage Actor to Critic) [33] and TRPO (Trust Region Policy Optimization) [51] represent the classical on-policy gradient methods with flattened state vector. The study aims to optimize task offloading decision in multi-user multi-server environment.
- D4PG [40]: An off-policy, multi-actor distributed model. This study aims to maximize the task success rate by considering communication interferences, resource allocation and delay requirements.
- TD3 [39]: It utilize DAG (Directed acyclic graph) based state representation with single actor and twin critics. It aims to adaptively allocate computational and network resources, reduce average service time, balance resource utilization, and minimize task delay rates.
- LE (Local Execution) that considers no task offloading and FOES (Full Offloading to Edge Server) where all the tasks are offloaded to the edge servers. These strategies provide the intuitive lower/upper bounds.

Section 5.3.1 presents the convergence comparison, Section 5.3.2 presents the impact of task failure rates during the computation and task failure rates during the communication. Section 5.3.3 presents the impact of varying user and edge server densities, Section 5.3.4 and Section 5.3.5 present a comparison based on varying base station configurations in terms of bandwidth and computational capability, respectively.

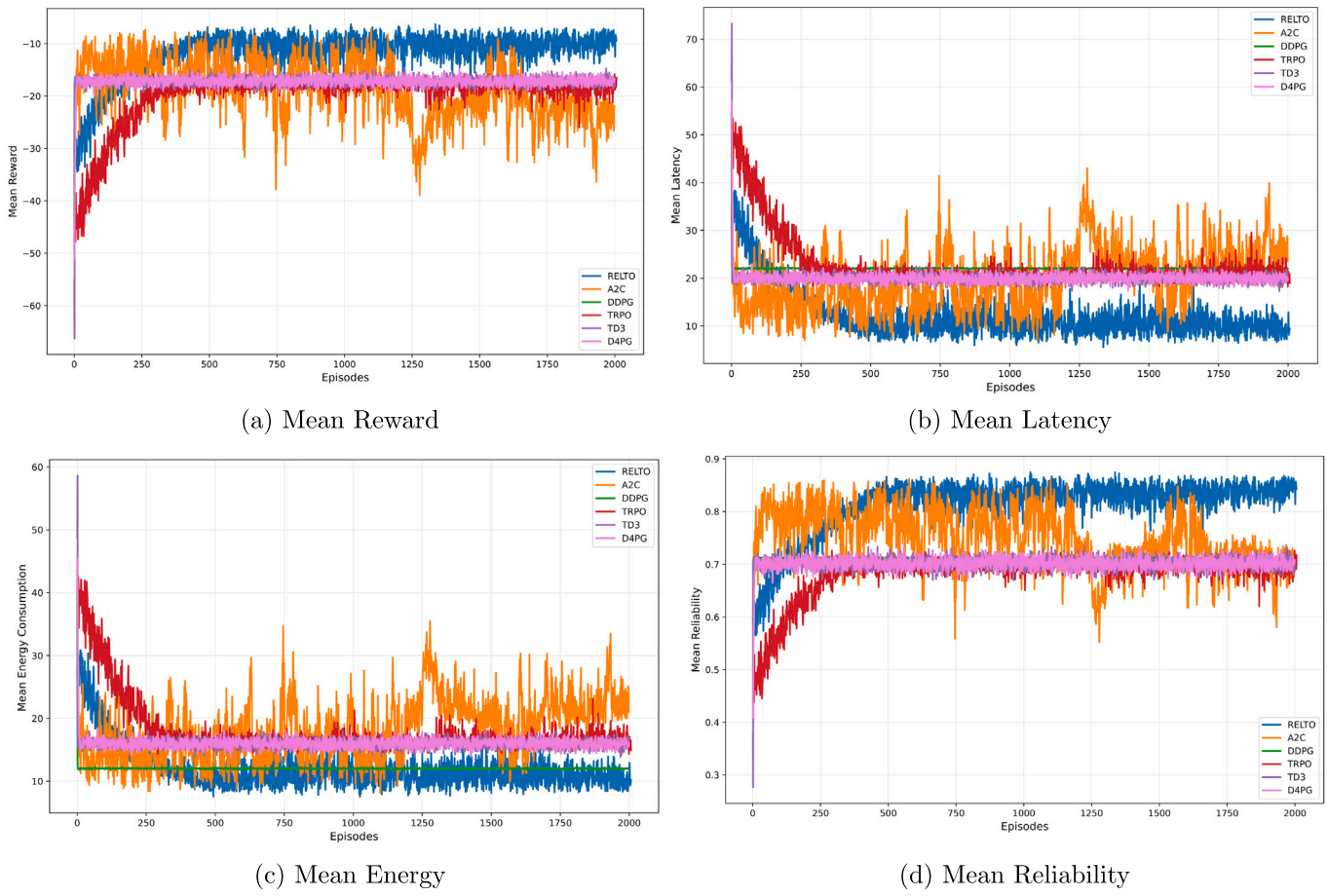


Fig. 7. Comparison of convergence performance over time.

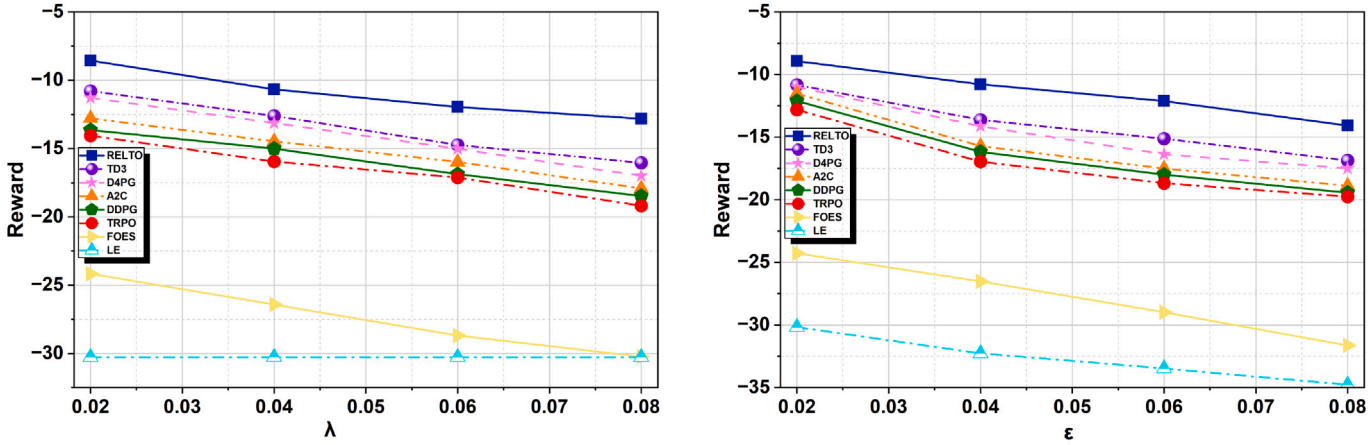
5.3.1. Convergence comparison

Fig. 7 presents the comparison based on the convergence performance over time. The benchmark methods converge faster initially, however reach a suboptimal solution. Their faster initial convergence stems from a simpler learning objective and less complex state representations. Whereas the convergence of the proposed algorithm increases over time, which ensures progressive learning, as it fine-tunes its offloading strategy over many episodes. RELTO feature attention module necessitates extended training to learn a truly discriminative global state vector from high-dimensional MEC observations. This deep contextual understanding, combined with the innovative reward-weighting attention module, allows RELTO to dynamically adapt its multi-objective reward function by prioritizing latency, energy, and reliability based on real-time environmental conditions. This continuous adaptation ensures that RELTO gradually optimizes across all the relevant metrics and keeps a balanced trade-off, ensuring the long-term optimality of the system performance.

5.3.2. Impact of task failure rates

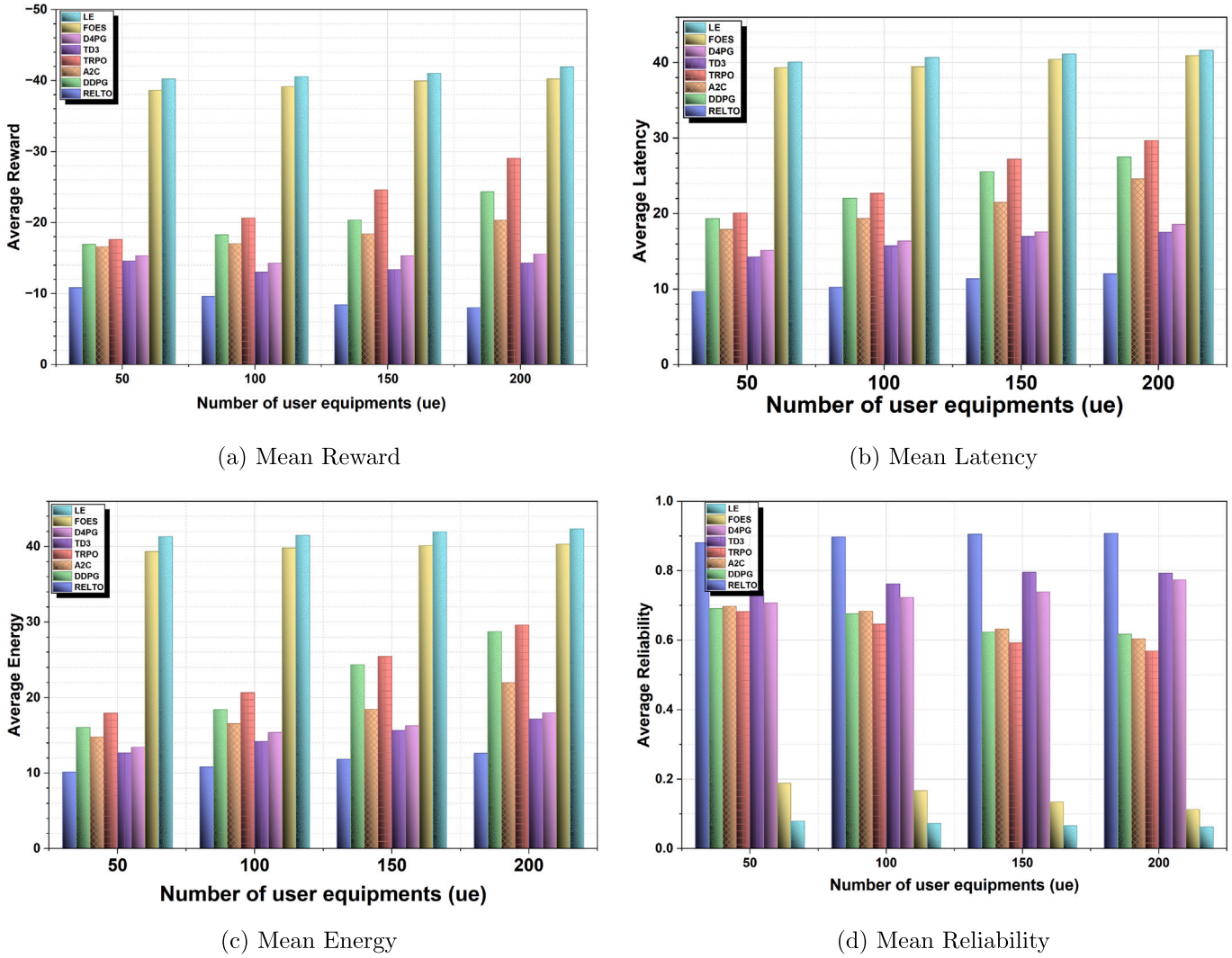
The task failure rates directly impact the system performance. Fig. 8 presents the impact of varying task failure rates on the reward function. Specifically, concerning task failures during communication λ, as depicted in Fig. 8(a), RELTO demonstrates remarkable stability. Its average reward remains consistently superior, around −8 to −12, as the communication failure rate λ increases from 0.02 to 0.08. This resilience is attributed to RELTO ability to utilize its attention mechanisms to detect deteriorating network conditions and then, guided by its reliability-prioritizing reward, choose more robust communication paths or carry out the local execution to ensure task completion. In contrast, other benchmark DRL algorithms such as TD3, initially showing good performance, experiences a gradual decline in reward from −11 at $\lambda = 0.02$ to −16 at $\lambda = 0.08$. D4PG follows a similar trend, with its reward decreasing from −12 at $\lambda = 0.02$ to −17 at $\lambda = 0.08$. However DDPG, A2C, TRPO exhibit greater instability, with their rewards significantly degrading around −14 to −20 as λ increases, indicating their struggle to manage increasing communication unreliability due to suboptimal offloading decisions and poor adaptation to changing network conditions. For fixed strategies, as the λ increases, LE remains relatively unaffected. For FOES, the frequency of failed offloaded tasks increases rapidly and causes a continuous reward decline, demonstrating the fragility of fixed offloading policies in unreliable network environments.

Similarly, regarding task failures during computation ϵ, illustrated in Fig. 8(b), RELTO reward starts at approximately −8 at $\epsilon = 0.02$ and shows a moderate decline to around −14 as the computation failure rate increases to 0.08. While a drop in reward is observed with increasing failure rates, this decline is notably less severe compared to other models. This robustness against computational failures highlights RELTO capacity, informed by its feature attention, to identify overloaded or less reliable edge servers and its reward weighting attention to prioritize choosing stable computational resources. This allows RELTO to adapt its offloading policy to mitigate computational stress, unlike other models. TD3 shows a decline, with its reward falling from −11 to −16. D4PG experiences a more significant drop, with its reward decreasing from −12 to −18 as ϵ increases. However, other models exhibit higher instability and more significant drops in reward around −11 to −20 due to inefficient resource allocation and poor task offloading decisions under increasing computational stress. Furthermore, for fixed strategies, increasing the computation failure rates ϵ, LE suffers due to its reliance on limited local processing capacity. For FOES, it offloads



(a) Task failure rates during communication (b) Task failure rates during computation

Fig. 8. Comparison based on varying task failure rates on reward.



(a) Mean Reward (b) Mean Latency

(c) Mean Energy (d) Mean Reliability

Fig. 9. Comparison based on varying user loads (ue).

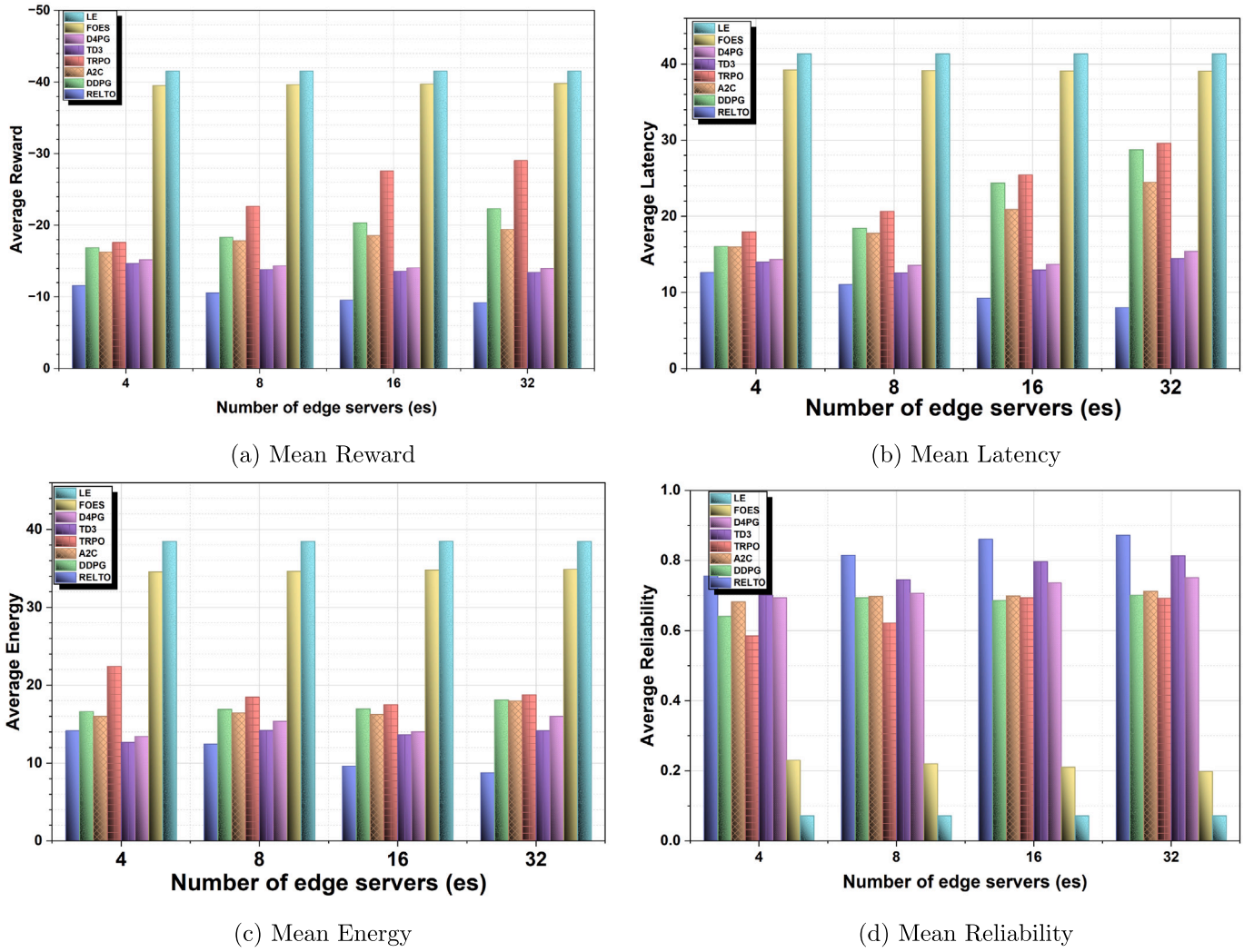


Fig. 10. Comparison based on varying edge devices (es).

the tasks to more edge servers and experiences a steady drop in reward as ϵ increases. This is attributed to edge-side computation bottlenecks that emerge when task volume increases, resulting in queuing delays, overload, and eventually task failures.

5.3.3. Impact of varying number of users and edge devices

Figs. 9 and 10 provide the impact of varying user loads (such as 100, 150, 200, and 250 *ue*) and varying numbers of edge servers (such as 4, 8, 16, and 32 es) on the performance metrics (reward, latency, energy, and reliability). As the number of user devices increases with other parameters remaining constant, it leads to an increase in the number of generated tasks. RELTO, because of its context-aware state space, shows superior performance in terms of reward over TD3, D4PG, A2C, DDPG, and TRPO, which struggle with high state space as the number of user equipments increases.

The average energy consumption and latency of RELTO remain consistently lower. At $ue = 200$, RELTO average energy consumption and latency is reduced by 10% and 15% compared to TD3 and D4PG. Meanwhile, the reliability of RELTO is also higher as compared to other strategies because of choosing optimal communications and computations resources to avoid task failures. For the fixed strategies the LE, where the tasks cannot be offloaded for computations and only processed locally, it results in higher energy consumption, higher latency, and less reliability. FOES latency and energy gets higher with an increasing number of user devices because of high server utilization and queuing delay causing bandwidth bottleneck. Similarly, when the number of *ue* is small and the edge server load is low, RELTO, A2C, DDPG and TRPO show significant performance improvements over the LE and FOES schemes.

Additionally, as the number of edge devices increases, RELTO effectively handles varying numbers of edge servers by decreasing the energy consumption by 10 J which indicates its efficiency in selecting the most energy-optimal offloading paths and balancing energy consumption between communication and computation, benefiting from better resource distribution. RELTO highlights its robust decision-making, where it intelligently distributes tasks and avoids congested servers, ensuring a higher task completion rate. Furthermore, the adaptive reward mechanism can prioritize reliability when resources are scarce or when faced with potential bottlenecks, which contributes to 10% higher reliability than TD3 and D4PG. The DRL-baseline strategies improve performance with more servers by learning to utilize the increased resources. However, managing a larger number of servers increases the action space complexity. Meanwhile, the performance of fixed strategies, such as LE remains constant and unaffected by ES count throughout the key objectives, while for FOES, the latency decreases as more servers become available. However, the average energy consumption increases with more active servers.

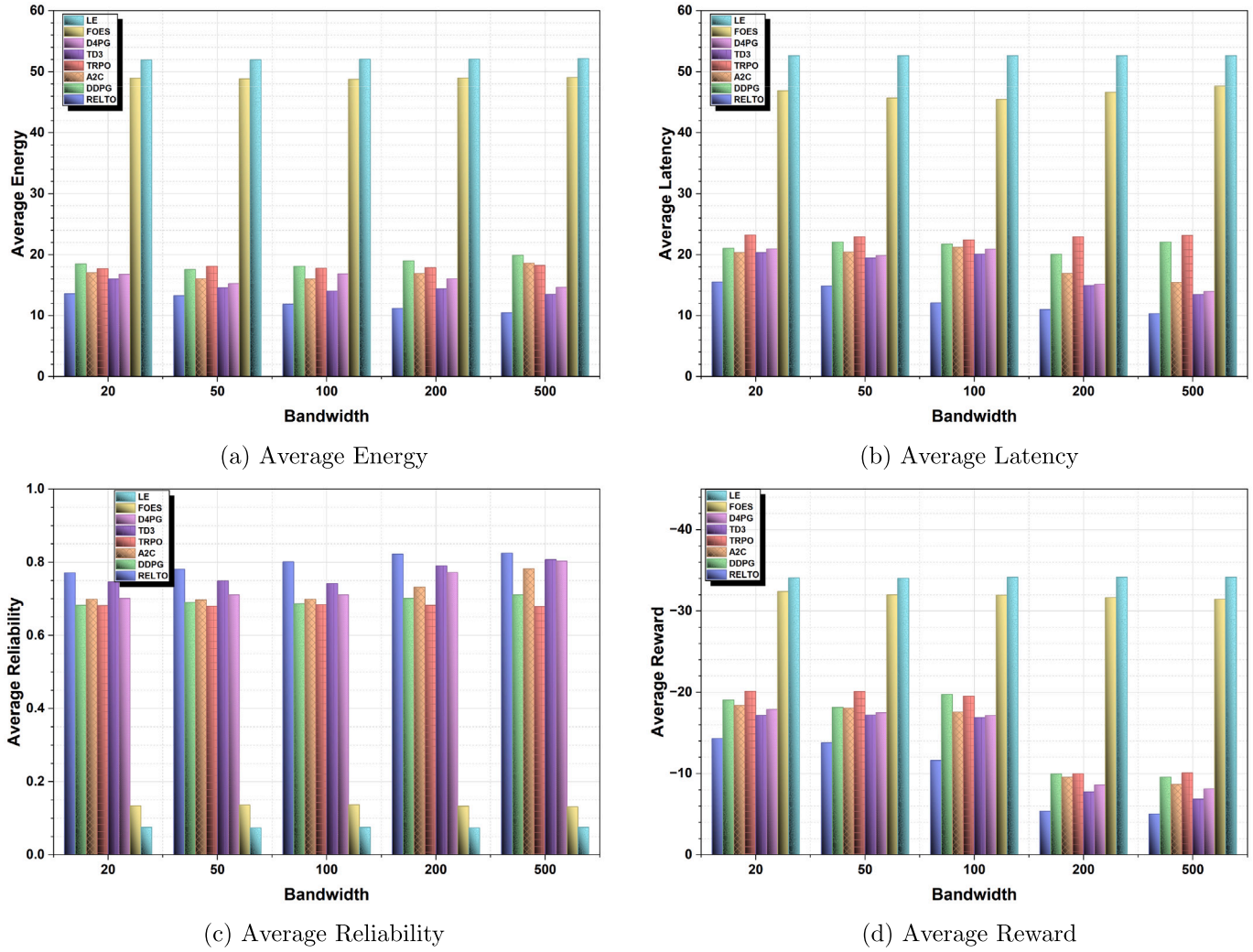


Fig. 11. Comparison based on varying bandwidth.

5.3.4. Impact of varying bandwidth

Fig. 11 illustrates the impact of varying bandwidth on key performance metrics. RELTO consistently achieves the lowest energy consumption across all bandwidths, ranging from approximately 13 J at 20 MHz to 10 J at 500 MHz, demonstrating its efficiency in balancing communication and computation energy. TD3 and D4PG show similar energy levels, 5%–10% higher than RELTO, while A2C, DDPG AND TRPO consumes notably more energy, particularly at lower bandwidths, though still outperforming fixed strategies.

In terms of latency, RELTO achieves the lowest average, decreasing from 16 ms at 20 MHz to 10 ms at 500 MHz. Its adaptive approach prioritizes low-latency paths at higher bandwidths and switches to local execution for smaller tasks when bandwidth is constrained. TD3, D4PG, TRPO, A2C, and DDPG also show decreasing latency with increasing bandwidth, however consistently perform worse than RELTO.

RELTO exhibits the highest reliability, increasing from 0.78 at 20 MHz to 0.85 at higher bandwidths, demonstrating its ability to adapt to varying network conditions and avoid unreliable offloading paths. TD3 and D4PG are particularly less reliable at lower bandwidths. RELTO also achieves the highest average reward, improving from −14 at 20 MHz to −4 at 500 MHz, reflecting its superior capability to optimize the multi-objective reward function by dynamically adjusting to bandwidth conditions. While TD3, D4PG,TRPO, A2C, and DDPG show improving rewards with bandwidth, they lag behind RELTO, with TRPO performing the worst at higher bandwidths due to less effective state representations and fixed reward functions.

For fixed strategies, LE remains unaffected by bandwidth, while FOES suffers from higher latency and energy consumption and lower reliability, highlighting the communication bottleneck inherent in static full-offloading strategies and their inability to handle dynamic network conditions.

5.3.5. Impact of varying computation capability of edge devices

Fig. 12 demonstrates the impact of varying computational capabilities of edge devices on key performance metrics. As base station computation capability increases, the RELTO feature attention effectively identifies more powerful resources, enabling its policy to offload tasks more efficiently and minimize latency by leveraging the increased processing speed. The adaptive reward mechanism inherently prioritizes latency when high-computation edge servers are available. In comparison, TD3, D4PG, TRPO, A2C, and DDPG show decreasing latencies with increasing BS capability however, consistently fall behind RELTO due to less effective task-to-server mapping, as these DRLs lack the granular state understanding provided by FeatureAttention.

With higher computational power, RELTO can complete more tasks within deadlines, with its adaptive reward mechanism prioritizing reliability when needed. TD3 and D4PG also show improvements in reliability but remain 5%–12% less reliable than RELTO. Faster computation reduces task completion time, which can decrease energy consumption by *es*. RELTO balances energy efficiency with other objectives, optimizing energy usage by leveraging the increased compute

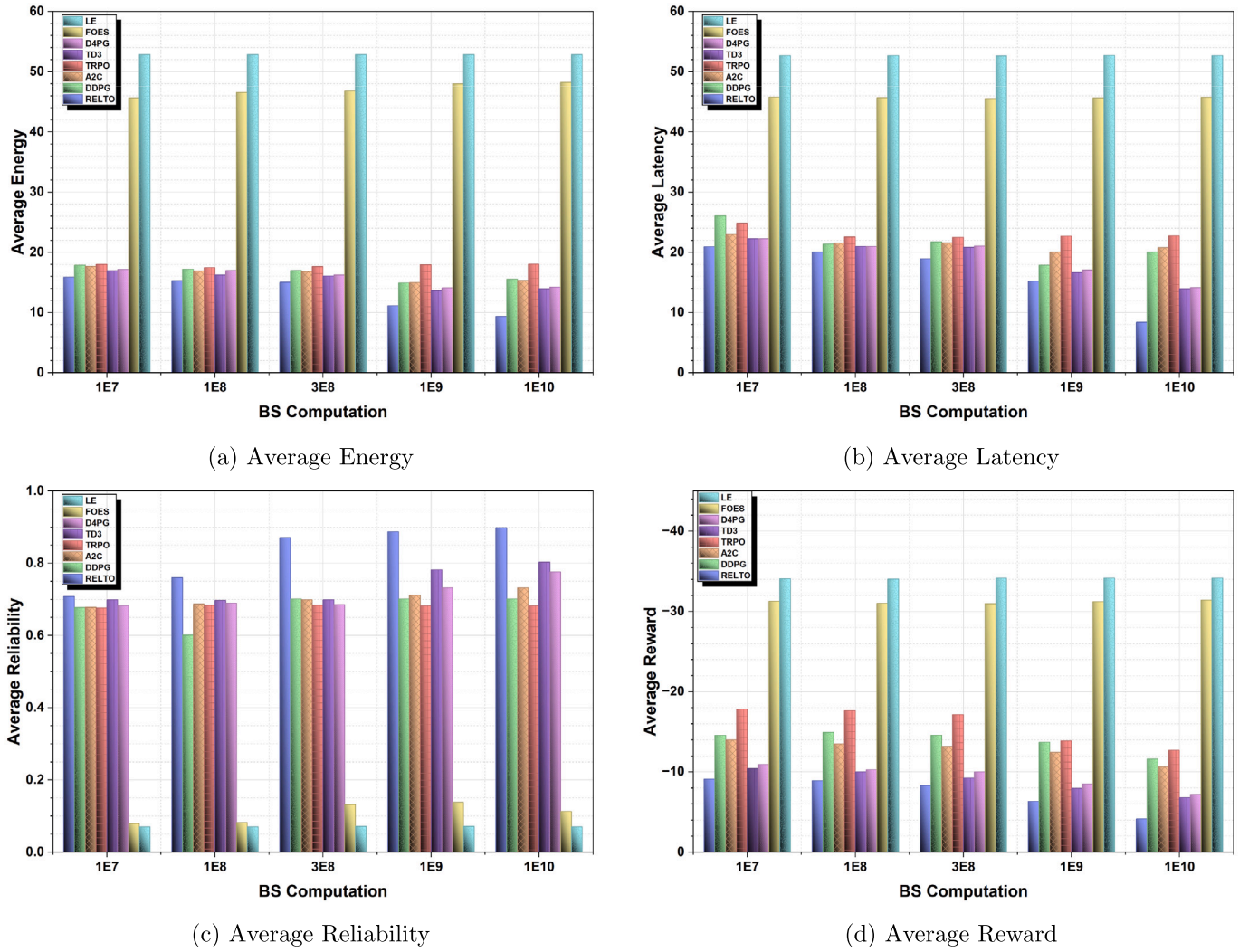


Fig. 12. Comparison based on varying edge server computations.

power. In contrast, other exhibit 5%–10% higher energy consumption than RELTO.

RELTO achieves the highest average reward, improving from approximately −9 at 1E7 FLOPs to −4 at 1E10 FLOPs. This is attributed to its adaptive reward weighting, which dynamically leverages increased computational power to optimize multi-objective performance. While other models also benefit from more resources, their static reward functions and less sophisticated state representations prevent them from achieving the same level of optimization as RELTO.

For fixed strategies, LE remains unaffected by BS capability, while FOES consistently underperforms across all metrics due to its lack of adaptability to network conditions and task characteristics, leading to queue overflows and task failures. Its inability to re-route or abort tasks further limits its effectiveness.

6. Conclusion

This study addresses the critical multi-objective optimization problem within dynamic MEC environments, specifically balancing task latency, energy consumption, and reliability. We introduce RELTO, a novel DRL algorithm built upon PPO. The core innovation is a context-aware adaptive reward weighting mechanism, empowered by dual attention networks that leverage an attention-driven feature extractor to dynamically prioritize conflicting objectives based on real-time environmental states. Complementing this, its hybrid action space enables granular offloading control.

Through extensive experimentation across diverse scenarios such as including varying network bandwidths, edge server densities, base station computational capabilities, and fluctuating communication and computation failure rates. RELTO conclusively demonstrates superior performance in achieving optimal trade-offs, consistently outperforming established Deep Reinforcement Learning baselines by simultaneously minimizing latency and energy consumption while maximizing task reliability, even under highly dynamic and unpredictable system loads.

The future direction of this research involves incorporating user mobility dynamics into the multi-objective optimization framework by integrating predictive mobility models within the RELTO state representation. This will enable proactive handover management and resource reallocation, ensuring seamless task continuity in highly dynamic network environments. Additionally, the framework will be extended to account for heterogeneous device capabilities and diverse task types with varying QoS requirements, thereby improving fairness and adaptability. Furthermore, the incorporation of security and privacy preserving mechanisms, such as lightweight federated learning, to safeguard sensitive data during task offloading. Finally, we plan

to develop a hybrid evaluation framework under more realistic MEC conditions to comprehensively validate the scalability, robustness, and practicality of RELTO in real-world deployments.

CRediT authorship contribution statement

Anam Nasir: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Data curation, Conceptualization. **Xiang He:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Methodology, Funding acquisition, Formal analysis. **Teng Wang:** Writing – review & editing, Formal analysis, Data curation, Conceptualization. **Haomai Shi:** Writing – review & editing, Conceptualization. **Zhongjie Wang:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (62372140), and the Postdoctoral Science Foundation of Heilongjiang Province, China (LBH-Z23145).

Appendix. Linearization and error bound for $\log_2(1+\text{SINR})$

To linearize the upload rate $U_{i,m,n}(t) = b\log_2(1+\text{SINR}_{i,m,n}(t))$ in Eq. (6), we approximate $f(\text{SINR}) = \log_2(1+\text{SINR})$ using a first-order Taylor expansion around a reference point SINR_0. We derive the expansion, provide the exact Lagrange remainder, and establish a conservative error bound.

Derivative and first-order expansion

The first derivative is:

$$f'(\text{SINR}) = \frac{1}{(1+\text{SINR})\ln(2)}$$

At SINR_0:

$$f'(\text{SINR}_0) = \frac{1}{(1+\text{SINR}_0)\ln(2)}$$

The first-order Taylor expansion is:

$$f(\text{SINR}) \approx f(\text{SINR}_0) + f'(\text{SINR}_0)(\text{SINR}-\text{SINR}_0) + R_1(\text{SINR})$$

where $R_1(\text{SINR})$ is the remainder. Substituting into Eq. (6) gives:

$$U_{i,m,n}(t) \approx b\left(\log_2(1+\text{SINR}_0) + \frac{1}{(1+\text{SINR}_0)\ln(2)}(\text{SINR}_{i,m,n}(t)-\text{SINR}_0)\right)$$

Second derivative and Lagrange remainder

The second derivative is:

$$f''(\text{SINR}) = -\frac{1}{(1+\text{SINR})^2\ln(2)}$$

The Lagrange remainder is:

$$R_1(\text{SINR}) = \frac{f''(\xi)}{2}(\text{SINR}-\text{SINR}_0)^2 = -\frac{1}{2(1+\xi)^2\ln(2)}(\text{SINR}-\text{SINR}_0)^2$$

where $\xi \in (\min(\text{SINR},\text{SINR}_0), \max(\text{SINR},\text{SINR}_0))$. The negative remainder indicates the approximation may overestimate the rate, with the magnitude depending on ξ.

Conservative absolute error bound

For a neighborhood $|\text{SINR}-\text{SINR}_0| \le \Delta$, we bound $|R_1|$ using the smallest denominator $(1+\xi)^2$. Since $\text{SINR} \ge 0$, let $\text{SINR}_{\min} = \max(0, \text{SINR}_0 - \Delta)$. The maximum $|f''(\xi)|$ occurs at $\text{SINR}_{\min}$:

$$|R_1(\text{SINR})| \le \frac{1}{2\ln(2)} \cdot \frac{\Delta^2}{(1+\text{SINR}_{\min})^2}$$

The rate error is:

$$|U_{\text{exact}} - U_{\text{approx}}| \le b \cdot \frac{1}{2\ln(2)} \cdot \frac{\Delta^2}{(1+\text{SINR}_{\min})^2}$$

such as, with $\text{SINR}_0 = 10$, $\Delta = 10$ $\text{SINR}_{\min} = 0$, and $b = 1$, the bound is:

$$\frac{1}{2\ln(2)} \cdot 10^2 \approx 72.13\,\text{bits/s/Hz}$$

which is conservative.

Data availability

No data was used for the research described in the article.

References

[1] S.K.U. Zaman, A.I. Jehangiri, T. Maqsood, Z. Ahmad, A.I. Umar, J. Shuja, et al., Mobility-aware computational offloading in mobile edge networks: a survey, Clust. Comput. (2021) 1–22.
[2] Y. Mao, C. You, J. Zhang, K. Huang, K.B. Letaief, A survey on mobile edge computing: The communication perspective, IEEE Commun. Surv. Tutorials 19 (4) (2017) 2322–2358.
[3] Y.C. Hu, M. Patel, D. Sabella, N. Sprecher, V. Young, Mobile edge computing—A key technology towards 5G, ETSI White Pap. 11 (11) (2015) 1–16.
[4] K. Shuai, Y. Miao, K. Hwang, Z. Li, Transfer reinforcement learning for adaptive task offloading over distributed edge clouds, IEEE Trans. Cloud Comput. 11 (2) (2022) 2175–2187.
[5] S.P. Panda, K. Ray, A. Banerjee, Dynamic edge user allocation with user specified QoS preferences, in: Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December (2020) 14–17, Proceedings 18 (187–197), Springer International Publishing, 2020.
[6] M.Y. Akhlaqi, Z.B.M. Hanapi, Task offloading paradigm in mobile edge computing-current issues, adopted approaches, and future directions, J. Netw. Comput. Appl. 212 (2023) 103568.
[7] L. Ale, S.A. King, N. Zhang, A.R. Sattar, J. Skandaraniyam, D3PG: Dirichlet DDPG for task partitioning and offloading with constrained hybrid action space in mobile-edge computing, IEEE Internet Things J. 9 (19) (2022) 19260–19272.
[8] Z. Cheng, M. Liwang, N. Chen, L. Huang, X. Du, M. Guizani, Deep reinforcement learning-based joint task and energy offloading in UAV-aided 6G intelligent edge networks, Comput. Commun. 192 (2022) 234–244.
[9] J. Qi, Y. Liu, Y. Ling, B. Xu, Z. Dong, Y. Sun, Research on an intelligent computing offloading model for the internet of vehicles based on blockchain, IEEE Trans. Netw. Serv. Manag. 19 (4) (2022) 3908–3918.
[10] J. Liu, A. Zhou, C. Liu, T. Zhang, L. Qi, S. Wang, R. Buyya, Reliability-enhanced task offloading in mobile edge computing environments, IEEE Internet Things J. 9 (13) (2021) 10382–10396.
[11] C.F. Liu, M. Bennis, M. Debbah, H.V. Poor, Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing, IEEE Trans. Commun. 67 (6) (2019) 4132–4150.
[12] L. Zhao, Y. Ren, K. Sakurai, Reliable workflow scheduling with less resource redundancy, Parallel Comput. 39 (10) (2013) 567–585.
[13] S. Pang, S. Wang, Joint wireless source management and task offloading in ultra-dense network, IEEE Access 8 (2020) 52917–52926.
[14] S. Lin, W. Liang, J. Li, Reliability-aware service function chain provisioning in mobile edge-cloud networks, in: 2020 29th International Conference on Computer Communications and Networks, ICCCN, IEEE, 2020, pp. 1–9.
[15] C. Li, J. Xia, F. Liu, D. Li, L. Fan, G.K. Karagiannidis, A. Nallanathan, Dynamic offloading for multiuser muti-CAP mec networks: A deep reinforcement learning approach, IEEE Trans. Veh. Technol. 70 (3) (2021) 2922–2927.
[16] C. Fang, Z. Hu, X. Meng, S. Tu, Z. Wang, D. Zeng, et al., Drl-driven joint task offloading and resource allocation for energy-efficient content delivery in cloud–edge cooperation networks, IEEE Trans. Veh. Technol. 72 (12) (2023) 16195–16207.
[17] J. Chen, P. Chen, X. Niu, Z. Wu, L. Xiong, C. Shi, Task offloading in hybrid-decision-based multi-cloud computing network: a cooperative multi-agent deep reinforcement learning, J. Cloud Comput. 11 (1) (2022) 90.

[18] X. Zhu, Y. Luo, A. Liu, M.Z.A. Bhuiyan, S. Zhang, Multiagent deep reinforcement learning for vehicular computation offloading in IoT, IEEE Internet Things J. 8 (12) (2020) 9763–9773.
[19] X. Liu, L. Feng, P. Zhang, Y. Yu, J. Wang, An efficient computational offloading method using deep reinforcement learning in edge-end-cloud, Ad Hoc Netw. (2025) 103941.
[20] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, et al., Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533.
[21] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, et al., Continuous control with deep reinforcement learning, 2015, arXiv preprint arXiv:1509.02971.
[22] J. Wang, M. Zhang, Q. Yin, L. Yin, Y. Peng, Multi-agent reinforcement learning for task offloading with hybrid decision space in multi-access edge computing, Ad Hoc Networks 166 (2025) 103671.
[23] N. Wilde, S.L. Smith, J. Alonso-Mora, Scalarizing multi-objective robot planning problems using weighted maximization, IEEE Robot. Autom. Lett. 9 (3) (2024) 2503–2510.
[24] M. Bowling, J.D. Martin, D. Abel, W. Dabney, Settling the reward hypothesis, in: International Conference on Machine Learning, PMLR, 2023, pp. 3003–3020.
[25] K. Freitag, K. Ceder, R. Laezza, K. Åkesson, M.H. Chehreghani, Sample-Efficient Curriculum Reinforcement Learning for Complex Reward Functions. arXiv preprint arXiv:2410.16790.
[26] Z. Liu, Z. Jia, X. Pang, DRL-based hybrid task offloading and resource allocation in vehicular networks, Electronics 12 (21) (2023) 4392.
[27] X. Hou, Z. Ren, J. Wang, W. Cheng, Y. Ren, K.C. Chen, H. Zhang, Reliable computation offloading for edge-computing-enabled software-defined IoV, IEEE Internet Things J. 7 (8) (2020) 7097–7111.
[28] L. Zhao, B. Li, W. Tan, G. Cui, Q. He, X. Xu, et al., Joint coverage-reliability for budgeted edge application deployment in mobile edge computing environment, IEEE Trans. Parallel Distrib. Syst. 33 (12) (2022) 3760–3771.
[29] A. Taghinezhad-Niar, J. Taheri, Reliability, rental-cost and energy-aware multi-workflow scheduling on multi-cloud systems, IEEE Trans. Cloud Comput. 11 (3) (2022) 2681–2692.
[30] X. Li, X. Lan, A. Mirzaei, M.J.A. Bonab, Reliability and robust resource allocation for cache-enabled HetNets: Qos-aware mobile edge computing, Reliab. Eng. Syst. Saf. 220 (2022) 108272.
[31] H. Liu, L. Cao, T. Pei, Q. Deng, J. Zhu, A fast algorithm for energy-saving offloading with reliability and latency requirements in multi-access edge computing, IEEE Access 8 (2019) 151–161.
[32] L. Dong, W. Wu, Q. Guo, M.N. Satpute, T. Znati, D.Z. Du, Reliability-aware offloading and allocation in multilevel edge computing system, IEEE Trans. Reliab. 70 (1) (2019) 200–211.
[33] C. Silva, N. Magaia, A. Grilo, Task offloading optimization in mobile edge computing based on deep reinforcement learning, in: Proceedings of the International ACM Conference on Modeling Analysis and Simulation of Wireless and Mobile Systems, 2023, pp. 109–118.
[34] F. Song, H. Xing, X. Wang, S. Luo, P. Dai, Z. Xiao, B. Zhao, Evolutionary multi-objective reinforcement learning based trajectory control and task offloading in UAV-assisted mobile edge computing, IEEE Trans. Mob. Comput. 22 (12) (2022) 7387–7405.
[35] F. Vhora, J. Gandhi, A. Gandhi, Q-TOMEC: Q-learning-based task offloading in mobile edge computing, in: Futuristic Trends in Networks and Computing Technologies: Select Proceedings of Fourth International Conference on FTNCT 2021, Singapore: Springer Nature Singapore., 2022, pp. 39–53.
[36] J. Liang, B. Ma, Z. Feng, J. Huang, Reliability-aware task processing and offloading for data-intensive applications in edge computing, IEEE Trans. Netw. Serv. Manag. 20 (4) (2023) 4668–4680, 80.
[37] P. Zhang, Y. Chen, M. Zhou, G. Xu, W. Huang, Y. Al-Turki, A. Abusorrah, A fault-tolerant model for performance optimization of a fog computing system, IEEE Internet Things J. 9 (3) (2021) 1725–1736.
[38] H. Ma, R. Li, X. Zhang, Z. Zhou, X. Chen, Reliability-aware online scheduling for DNN inference tasks in mobile-edge computing, IEEE Internet Things J. 10 (13) (2023) 11453–11464.
[39] S. Wang, B. Yu, N. Wang, W. Wang, Research on TD3-based offloading strategies for complex tasks in MEC systems, in: 2024 10th International Conference on Computer and Communications, ICCC, IEEE, 2024, pp. 194–201.
[40] Q. He, Q. Li, C. Zhang, X. Wang, Y. Bi, L. Zhao, et al., Task optimization allocation in vehicle based edge computing systems with deep reinforcement learning, IEEE Trans. Comput. (2025).
[41] N. Gholipour, M.D. de Assuncao, P. Agarwal, J. Gascon-Samson, R. Buyya, TPTO: A transformer-PPO based task offloading solution for edge computing environments, in: 2023 IEEE 29th International Conference on Parallel and Distributed Systems, ICPADS, IEEE., 2023, pp. 1115–1122.
[42] F. Zhou, L. Zhao, X. Ding, S. Wang, Enhanced DDPG algorithm for latency and energy-efficient task scheduling in MEC systems, Discov. Internet Things 5 (1) (2025) 40.
[43] Q. Liu, Z. Tian, N. Wang, Y. Lin, DRL-based dependent task offloading with delay-energy tradeoff in medical image edge computing, Complex & Intell. Syst. 10 (3) (2024) 3283–3304.
[44] G.K. Walia, M. Kumar, Computational offloading and resource allocation for iot applications using decision tree based reinforcement learning, Ad Hoc Networks 170 (2025) 103751.
[45] J. Ma, B. Shang, H. Song, Y. Huang, P. Fan, Reliability versus latency in iIoT visual applications: A scalable task offloading framework, IEEE Internet Things J. (2022).
[46] P. Yao, X. Chen, Y. Chen, Z. Li, Deep reinforcement learning based offloading scheme for mobile edge computing, in: 2019 IEEE International Conference on Smart Internet of Things (SmartIoT), IEEE., 2019, pp. 417–421.
[47] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint arXiv:1707.06347.
[48] Z.H. Meybodi, A. Mohammadi, M. Hou, E. Rahimian, S. Heidarian, J. Abouei, K.N. Plataniotis, Multi-content time-series popularity prediction with multiple-model transformers in MEC networks, Ad Hoc Netw. 157 (2024) 103436.
[49] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, et al., Attention is all you need, Adv. neural inf. process. syst. vol. 30 (2017).
[50] S. Huang, S. Ontañón, A closer look at invalid action masking in policy gradient algorithms, 2020, arXiv preprint arXiv:2006.14171.
[51] J. Schulman, S. Levine, P. Abbeel, M. Jordan, P. Moritz, Trust region policy optimization, in: International Conference on Machine Learning, PMLR., 2015, pp. 1889–1897.



Anam Nasir received her M.S. degree in Software Engineering from the Department of Computer Science, COMSATS University Islamabad, Pakistan, in 2021. She is currently pursuing her Ph.D. degree in Software Engineering at the Harbin Institute of Technology, China. Her research interests include edge computing, edge intelligence, and deep learning.



Xiang He received his Ph.D. degree from the School of Computer Science and Technology, Harbin Institute of Technology in 2023. He is currently an assistant professor at the Faculty of Computing, Harbin Institute of Technology, Harbin, China. His research interests include edge computing, microservice system infrastructure design, and self-adaptive systems.



Teng Wang received the B.S. degree in Software Engineering from Harbin Institute of Technology in 2021 and the M.S. degree in Software Engineering from the same university in 2023. He is currently pursuing his Ph.D. degree in Software Engineering at Harbin Institute of Technology, China. His research interests include distributed computing, edge intelligence, and edge computing.



Haomai Shi received her B.S. degree from the School of Computer Science and Technology, Huaqiao University in 2021. She is currently pursuing her Ph.D. degree in Software Engineering at Harbin Institute of Technology, China. Her research interests include edge computing, self-adaptive microservice systems, and computing system simulation.



Zhongjie Wang is a professor at the Faculty of Computing, Harbin Institute of Technology (HIT). He received the Ph.D. degree in Computer Science from Harbin Institute of Technology in 2006. His research interests include services computing, mobile and social networking services, and software architecture. He is the author of more than 100 publications and a member of the IEEE.